Let's Learn ES6

WE'LL COVER THE FOLLOWING

- How did we get here?
- What happened to ES4?
  - Proposal Stages
- What we will cover
- The future is bright
- Special Thanks
- Additional resources

How did we get here?

How did we end up with ES6? A lot of things had to happen in order for ES6 to come around. For a long time, JavaScript was a fairly stagnant language. There were many years between new releases. Because of that, people seemed to get used to the quirks and nuances of the language, and as developers we made up for areas that JavaScript seemed to lack.

However people wanted new features and compile to JavaScript languages started to pop up. Languages like Coffeescript started to push features that would later influence features in ES6.

What happened to ES4?

You might have noticed, if you do a little digging, that there is no mention of an ES4 release. This is because it was completely abandoned [(1)], in 2007 Brendan Eich(JS creator) and Chris Wilson(Microsoft) could not come to an agreement around the features of the release. The proposed features would be a breaking change to many sites on the web. Because of this and many other issues the release was abandoned.

In 2008 however, progress started again, a new project called ECMAScript Harmony was created and it included many of the features we would have seen in ES4. There is a committee called TC39, Technical Committee 39, which drives the ECMAScript standard forward. This committee worked Harmony to create ES5, and it continues to work on each new version of the language.

Proposal Stages

In order for a feature to be added to the language it has to go through four stages. The stages consist of a strawman(the initial introduction), proposal, draft, candidate, and finally a finished product. Through out the stages the TC39 committee will be able to review the proposal, and anyone can review an in process proposal. To find out more check out the link in the additional resources section below [(2)].

What we will cover

In this course we will be covering quite a few features in the language. We will be looking at new syntax like generator functions and template literals, we can see how arrow functions can be used to our advantage. We will explore promises, and modules, we see how we can use them to make our code clean and readable. We will look at these features and much more!

The future is bright

JavaScript is one of the most popular languages in the world. It is being run just about everywhere, on servers and being used to control micro controllers. Platforms like Node.js are taking it to new places, now more than ever really learning JavaScript is going to be an important step in your career!

Special Thanks

A lot of people helped me get to this point in the project. Huge thanks to the people that have helped me with this course, Erin McPhee, Simon Bloom, Dustin Ruetz, and Nadia Rasul. With out their help this course would be an English students nightmare! And you know what, it still might be!

Additional resources

- 1:https://en.wikipedia.org/wiki/ECMAScript#4th_Edition_.28abandoned.29
- 2:Proposal stages https://docs.google.com/document/d/1QbEE0BsO4lvl7NFTn5WXWeiEIBfaVUF7Dk0hpPpPDzU/edit

State of ES6 support

WE'LL COVER THE FOLLOWING

- What is transpiling?
- Additional Resources

As mentioned in Chapter 1, ES6 was finalized in 2015. Since then, browsers have been rolling out support with each incremental update. At the time of writing (March 2016), the current version of Chrome (Chrome 49) has 91% support. The current version of Chrome Canary (a nightly build of Chrome) has 94% support.

While this is great for Chrome, not all browsers currently offer the same level of support. Firefox is pretty close at 91%, and Edge 14 is 86%. But, what about Internet Explorer 11? Unfortunately, IE 11 is at 16%, and it will never get higher than that.

If you want to find out what the level of support is for each individual browser, check out the Kangax Compatibility Table [1]. It is a great reference for which browsers or environments support what.

Looking at the table, you will see that most browsers are improving their support of ES6 features. Unfortunately, some are not, and even worse, some will never change. We can not always bank on users only using something like Chrome or Firefox. As developers, we need to make sure that we are supporting the widest range of browsers we need to for our projects. In order to do this, we need to take our ES6 code and transpile it down into ES5 JavaScript that will be readable by the browser.

What is transpiling?

Transpiling is the act of taking one language and compiling it to another

language. It is also referred to as source-to-source compiling. Let's take this example:

```
const numbers = [1,2,3,4,5];
const max = Math.max(...numbers);
```

In the above code, we use the spread operator to expand out the `numbers` array into `Math.max`. We will look at the spread operator in more detail later, but for now let's focus on the fact that this code will not run in any browser that do not support ES6. However, taking this code and running it through a transpiler, for example Babel [2], will compile it down to something that looks like this:

```
"use strict";

var numbers = [1, 2, 3, 4, 5];
var max = Math.max.apply(Math, numbers);
```

This is code that all browsers understand today. The idea of transpiling code is not new; many languages compile from one source to another. There are many languages [3] that use JavaScript as their compile target. One of the most popular is CoffeeScript [4]. This is a language that came about as a way to extend and add new functionality to JavaScript, so that you can write something like:

```
multi = (mul,numbers...) ->
   numbers.map(n -> n * multi)
```

That would equate to compiled JavaScript that looks like this:

```
var multi,
  slice = [].slice;

multi = function() {
  var mul, numbers;
  mul = arguments[0], numbers = 2 <= arguments.length ? slice.call(arguments, 1) : [];
  return numbers.map(n(function() {
    return n * multi;
  }));
};
```

CoffeeScript brought some elegant new features to JavaScript, and ultimately some of those features were the inspiration for features like arrow functions

in ES6!

We have talked about the current state of support and how we can use transpilers, or source-to-source compilers to work with ES6 in browsers that do not support it fully yet. In the next chapter, we will look at how to actually perform the transpile step!

Additional Resources

- 1:https://kangax.github.io/compat-table/es6/
- 2: https://babeljs.io/
- 3: https://github.com/jashkenas/coffeescript/wiki/list-of-languages-that-compile-to-js
- 4: http://coffeescript.org/

Using ES6 now

WE'LL COVER THE FOLLOWING

- Babel
- Babel CLI
- Babel Presets
  - Folder structure
- NPM Script
  - babelrc
- Gulp
  - Getting started with Gulp
    - gulpfile.js
  - The js task
- Additional Resources

While support for ES6 is increasing for evergreen browsers such as Chrome and Firefox, we can't always assume that users will be on one of those browsers. So, in order to utilize ES6 features now and make sure we won't run into cross-browser compatibility issues, we need to transpile our code.

Let's look at two possible ways we can perform the task of transpiling our code. First, we will use npm scripts and babel, and for the second, we will look at using gulp with babel.

Babel

Babel [(1)] is the go-to transpiler for ES6. It was originally called 6to5, but was later renamed to Babel as it was apparent that the name would not work moving forward [(2)]. With the release of Babel 6, the focus turned more towards making Babel pluggable. It created a system that allows you to create

plugins to transform your code! The default action for Babel in version 6 is not

to transform your ES6 to ES5 anymore, so you now have to include various presets.

The presets in Babel allow you to either pick and choose the transformations that you want, or you can select the `babel-preset-es2015` preset and work with all the features.

Babel CLI

In order to work with the two methods that we will look at, it is important to make sure you have Node.js installed. The easiest method would be to head over to the Node.js website and download the latest version for your operating system.

If everything goes as planned, you should have `node` available to you in your terminal. To confirm that Node.js is installed on your system, open your terminal and type `node -v` into the prompt.

```
$ node -v
v5.2.0
```

If you get a response that looks something like what you see above, you are good to go! If you are not super comfortable with the command line, I suggest checking out commandlinepoweruser.com [(3)] by Wes Bos. It's a great free video series to help you quickly get up and running with the common commands.

Once we have Node up and running we need to install the Babel CLI. To do so we will use npm. To test this, create a project folder and navigate there in the terminal. To get started we need to create a `package.json` file. To get this started with npm, we can run:

```
npm init
```

This will walk you through a few questions: what is the name of your project, version, description, etc. When it asks you about the `entry point`, you can leave it as `index.js` for now. Typically this would be the main file for your

project. If you know ahead of time what that will be, add it there.

Once you are done with these steps, a new `package.json` file will have been created. Now, it's time to install the babel package we need.

```
npm install --save-dev babel-cli
```

After running this line you will see a few new changes in your project directory. First, you will notice that there is now a `node_modules` folder, and second, that there is a new entry inside your `package.json` file. There is one key part to the code above: the `--save-dev` flag. This flag will save the module, `babel-cli`, in your package.json file under `devDependencies` section.

```
"devDependencies": {
  "babel-cli": "^6.6.5"
},
```

This section in the file is used for modules that will be used in development, meaning they are not needed in production. If we had run our install with `--save` instead, it would put the module under the `dependencies` section.

But, what is the point of all these sections? If you are unfamiliar with npm and the `package.json` file, it's a file that you can take from project to project and run `npm install` and anything listed in `devDependencies` or `dependencies` will be installed for that project. If you wanted to install just the `dependencies` you could run:

```
npm install --production
```

There will be cases when you are developing your apps that you will want to use certain modules exclusively for development, and certain modules exclusively for production. Using `--save` and `--save-dev` you can put those modules in the right buckets.

Babel Presets

As mentioned earlier, Babel does not default to transpiling ES6 right off the bat. We have to include the `babel-preset-es2015` module as well.

```
npm install --save-dev babel-preset-es2015
```

This module ensures that we have the ability to transpile our code.

Folder structure

Before we go too far, lets talk about folder structure. If you are going to be following along, I have set up my folder structure as follows:

```
- project
    - src
      - app.js
    - public
    - package.json
```

In the root `project` folder I have a `src` folder and a `public` folder. The `src` folder is used to hold my pre-transpiled code. This would be the code that is written with the nice ES6 syntax, while the `public` folder will be the destination for our transpiled code. The `public` folder would also hold the CSS, HTML, and images that your app requires.

NPM Script

The first method of transpiling we will look at is using NPM to run Babel. In the `package.json` file there is a section called `scripts` where we can define commands we would like to run. Using the Babel CLI we can set up a script to compile our code and output it where we would like. The `scripts` section takes a JSON object as a value. The keys in this object will be the name of our script and the value will be the command to run.

```
"scripts": {
  "js" : "babel src/app.js --out-file public/app.js"
}
```

Adding the above to our `package.json` in the `scripts` section will take our code from `src/app.js`, run it through Babel, and output the file in `public/app.js`. The Babel command has many options to it that we will look at later, but the `--out-file` flag is used to determine the destination of the compiled source.

Go ahead and add some ES6 code to the `src/app.js` file. If you don't know

what to add, try this:

```
const person = {
  name: 'Ryan Christiani'
};

const testString = `My name is ${person.name}`;
```

If you type `npm run js` into your terminal you should have an output of something like this:

```
> testfiles@1.0.0 js /Users/yourusername/path/to/project
> babel ./src/app.js --out-file ./public/app.js
```

Now, the moment of truth: let's look in the `public/app.js` file. Note that if you did not create one before, Babel will go ahead and create it for you.

```
const person = {
  name: 'Ryan Christiani'
};

const testString = `My name is ${person.name}`;
```

Huh... this looks exactly the same. That is because we have not utilized the `es2015` preset yet!

babelrc

In order to tell Babel that we want to use presets that we have downloaded, we need to create a configuration file. This file is called `.babelrc`. If you have never worked with a dot-file before, you might think that it is a bit odd! Dot-files are meant to be hidden files and are typically used to store configuration information, or just to keep things hidden. By default you cannot see dot-files on your computer, however a quick Google search will show you how to turn them on for your finder. Since we are on the terminal, we can use the `ls -a` command to show any file or folder that starts with a `.`.

Sublime Text (the editor I am using throughout this book) or Atom will show you these files if you open your project folder in the editor.

In order to configure Babel to use our preset, we need to add this code to the `.babelrc` file.

```
{
  "presets": ["es2015"]
}
```

Babel configuration files take a JSON object with a `presets` key that contains an array of the presets you would like to use. There are many presets you can use, check out the Babel docs [4] for more information. Now, if you run `npm run js` it will properly transpile our code, changing the output to look like this:

```
'use strict';

var person = {
  name: 'Ryan Christiani'
};

var testString = 'My name is ' + person.name;
```

This is great, however let's make our script a little more robust. Right now, when we run the script, it will output the file and stop. If you want to keep working on the file we need to run this over and over again. We can configure the script to watch over the files and run when they change!

```
"scripts": {
  "js": "babel ./src/app.js --watch --out-file ./public/app.js"
}
```

By adding the `--watch` flag, we are telling Babel to listen for any changes in the `src/app.js` file, and when changes are made we want to output a transpiled version of that code to `public/app.js`. There are plenty more things you can do with npm scripts and the Babel CLI [5], however let's see how we can use Gulp [6] and Babel to transpile our code!

Gulp

Gulp is a popular task runner that helps to automate your workflow. There are a ton of different task runners out there, including Grunt, Broccoli, and Brunch. For this book, we will look at how to configure Gulp to automate working with Babel, and later on in the book, we will use this to work with modules as well. So lets get started!

Getting started with Gulp

In order to get started with Gulp, we need to install it globally on our system. Gulp is another npm module, so we install it like such: `npm install --global gulp-cli`. When working with gulp we have to install it globally only once, and also locally in each new project. We can do this by running: `npm install --save-dev gulp`.

gulpfile.js

To configure gulp we must first create a new file called `gulpfile.js`; this will be used to define our tasks. If you have never worked in Node before, you might not be familiar with the `require` function. This is a function that will pull in any modules from the `node_modules` folder, or your own modules. Since we ran `npm install --save-dev gulp` that placed it in our `node_modules` folder. In the `gulpfile.js` we can get access to `gulp` by requiring it.

```
const gulp = require('gulp');
```

This will store Gulp on a new variable called `gulp`. You will notice that I used `const` here! This `gulpfile.js` will be our first JavaScript file that uses ES6. I won't be going into much detail on the ES6 features we use here, that's what this book is for! We will discuss these features in more depth later throughout this book.

Gulp is all about tasks. We will start by defining some simple tasks in this file.

```
const gulp = require('gulp');

gulp.task('js', () => {

});

gulp.task('default', () => {

});
```

The `() =>` above is the syntax for an ES6 arrow function. Don't worry if you aren't familiar with this - we will discuss it later.

The two tasks we have created will work together, and also separately if we would like. The first of the tasks is one we will call `js` (note that you can call a

task anything you want with the exception of `default`). The `js` task will deal

with taking our JS and running it through Babel. In order to run this task we can run `gulp js` in our terminal. Of course, we have to define the task first.

The second task is called `default`. This is a special task: in here we can set up code that will kick off many other tasks, or in our case create a listener for when certain files change. To run this task we can simply type `gulp` in the terminal. Gulp knows that if it just sees `gulp` it will run the `default` task.

The js task

Before we get started we have to install a module the will allow babel to work with gulp. Oddly enough this module is called `gulp-babel`. Type `npm install --save-dev gulp-babel` into your terminal to start downloading the `gulp-babel` package.

In our gulp file, let's add these lines:

```
const gulp = require('gulp');
const babel = require('gulp-babel');

gulp.task('js', () => {
    return gulp.src('./src/app.js')
        .pipe(babel({
            presets: ['es2015']
        }))
        .pipe(gulp.dest('./public'));
});

gulp.task('default', () => {

});
```

The first thing we do is to require `gulp-babel`, then in our `js` task we return `gulp.src('./src/app.js')`. We return this to let gulp know that this will be an asynchronous event. The `.src` method lets us set the path for our file, and in this case it is a single file. When we get to the modules portion of this book we will change this to be a bit more robust. Gulp works with streams, meaning that we can keep moving the data down the pipe, so using the `.pipe` method we take the source files and send them through babel.

Just like earlier, we need to specify the presets we want to use. To do this, we can pass an object to the babel function to tell the babel plugin which preset

we would like to use. Our source gets transformed into browser-ready code,

and it gets passed along again, so we can pipe it into `gulp.dest`, which sets the final destination.

Now that our task is complete, running `gulp js` in the terminal should produce this output!

```
[19:50:35] Using gulpfile ~/Sites/lets-learn/book/testfiles/gulpfile.js
[19:50:35] Starting 'js'...
[19:50:35] Finished 'js' after 102 ms
```

If you don't have any errors, then everything should be good. Gulp will have run, taken your `app.js` from the `src` folder, transformed it, and saved it to the `public` folder.

You might have noticed that this only ran once. If we change some code, we have to run it again. Let's set up the ability for gulp to keep watching our files.

```
gulp.task('default', ['js'],() => {
    gulp.watch('src/app.js',['js']);
});
```

Altering the `default` task we add an additional argument, an array, for the tasks that we want to fire when we first use the task. The `gulp.watch` method takes a few arguments: a file path that waits for file changes, and an array of the task(s) you want to run. Now we can just run `gulp`.

```
$ gulp
[21:26:50] Using gulpfile ~/Sites/lets-learn/book/testfiles/gulpfile.js
[21:26:50] Starting 'js'...
[21:26:50] Finished 'js' after 102 ms
[21:26:50] Starting 'default'...
[21:26:50] Finished 'default' after 6.73 ms
```

Changing your `src/app.js` file will run the `js` task again! Let's get into learning about ES6 so that we can use this file. In a later chapter we will add to this file.

Additional Resources

- 1:https://babeljs.io/

- 2:https://www.youtube.com/watch?v=rKuNbEwoQfQ

- 3:http://commandlinepoweruser.com/
- 4:http://babeljs.io/docs/plugins/#presets
- 5:https://babeljs.io/docs/usage/cli/
- 6:http://gulpjs.com/

Numbers and Math in ES6

WE'LL COVER THE FOLLOWING

- Binary Literals
  - Math with binary numbers
- Octal Literals
  - Math with octal numbers
- Math
- Math.trunc

In ES6 there are a bunch of additions to Numbers and the Math object. There are new Number literals, Binary and Octal, as well a ton of new methods for `Math`.

Binary Literals

In ES6 we have to ability to create binary literals. In order to do this we need to prefix our binary number with `0b` or `0B`.

```
console.log(0b001) // The number 1
console.log(0b010) // The number 2
console.log(0b011) // The number 3
console.log(0b100) // The number 4
```

If you have never worked with binary numbers before, the pattern goes like this: Starting for the right to the left, the smallest number is as far right as possible. For example `0001` in binary is `1`.

8	4	2	1
0	0	0	1

If we wanted to get the numbers `3` it would be `0011`, adding `2` and `1`.

8	4	2	1
0	0	1	1

Let's do one more, how about `10`? Well that would be `1010` or `8` + `2`.

8	4	2	1
1	0	1	0

Math with binary numbers

We can perform mathematical operations with the binary numbers just like we normally would.

```
console.log(0b100 + 0b010)
```

Can you guess what that adds up too? Well `6` of course, the neat thing is we can add binary and integer numbers together.

```
console.log(0b110 + 9)
```

This will produce `15`.

Octal Literals

Just like binary literals we are able to create octal literals as well. Octal numbers are a base 8 number system, binary is a base 2 system. In Octal numbers each place is a multiple of 8, and the number present in that place will be used to determine the value. Let's look at some examples

512	64	8	1
0	0	0	0

Like the binary literals we use `0o` or `0O` to denote it what number type it will be, so `0o0001` will be the number `1`.

512	64	8	1
0	0	0	1

If we had `0o0011` we would get `9`. Now we don't just have to put a `0` or `1` in the place here, we can use a number from `0` to `7`.

So if we had `0o0020` this would be `16`, as it takes the multiple in that place and times it by `2`.

Math with octal numbers

Just like the binary literal, octals can be used to preform mathematical operations on with mixed number types.

```
console.log(0o0020 + 7) // 23
```

We can even mix it with binary numbers.

```
console.log(0o0010 + 0b0111) // 15
```

Math

In ES6 there are a ton of new Math methods. These include `Math.clz32`, `Math.imul`, `Math.sign`, `Math.log10`, `Math.log2`, `Math.log1p`, `Math.expm1`, `Math.cosh`, `Math.sinh`, `Math.tanh`, `Math.acosh`, `Math.asinh`, `Math.atanh`, `Math.trunc`, `Math.fround`, `Math.cbrt`, `Math.hypot`...woof that is a lot! A lot of the additions here are for Trigonometry.

We won't look at all of these, but I would like to show you one method, that is `Math.trunc`.

Math.trunc

The `.trunc()` method is used to truncate the number provided to it, for example:

```
console.log(Math.trunc(5.6) )//5
console.log(Math.trunc(9.231)) //9
```

It is used to just cut off the numbers after the decimal, this differs from `Math.floor`, `Math.ceil` and `Math.round` in that it just cuts them off, it does not perform any rounding on the number.

Strings, Regex & Unicode

WE'LL COVER THE FOLLOWING

- String Methods
  - `.startsWith`
  - `.endsWith`
  - `.includes`
    - Start position
  - `.repeat`
- Unicode
- Regex
  - Sticky match
  - Additional Resources

String Methods

Recently introduced in JavaScript are a few string methods that are more convenient methods for working with strings. These methods include `.startsWith`, `.endsWidth`, `.includes`, and `.repeat`.

Previously if we wanted to search for the existence of some text in a string we would have to use regex. Now there are a few methods that help us make this a bit easier.

`.startsWith`

If you wanted to see if a string started with a specific bit of text, we could have created a regular expression that looked like this.

```
const truth = "JavaScript is a really fun language!";
console.log(truth.match(/^JavaScript/)); //[ 'JavaScript',
```

```
                                        //index: 0,
                                        //input: 'JavaScript is a really fun language!' ]
console.log(truth.match(/^fun/)); //null
```

This regular expression would search for the work `JavaScript` at the beginning of our string. With the `.startsWith` method, we cause do that same thing.

```
const truth = "JavaScript is a really fun language!";

console.log(truth.startsWith("JavaScript")); //true
console.log(truth.startsWith("fun")); //false
```

The method will return `true` or `false` if the text you are looking for appears in the at the start of the string.

`.endsWith`

Similar to `.startsWith` the `.endsWith` method will check to see if a string ends with a specific bit of text. If we wanted to do this in with a regular expression we could match it like this.

```
const truth = "JavaScript is a really fun language!";
console.log(truth.match(/language!$/)); //['language!', index: 27 ....]
console.log(truth.match(/JavaScript$/)); //null
```

With the `.endsWith` method will allow us to make this check without having to write any regex, and it will simply return `true` or `false` after the check.

```
const truth = "JavaScript is a really fun language!";

console.log(truth.endsWith('language!')); //true
console.log(truth.endsWith('JavaScript')); //false
```

`.includes`

If we can check weather a string starts or ends with some text we should also be able to check if it includes some text. The `.includes` method will allow us to do just this. This is again another convenience method for something we could perform with a regular expression.

```
const truth = "JavaScript is a really fun language!";
console.log(truth.match(/fun/g)); //["fun"]
console.log(truth.match(/a/g)); //["a", "a", "a", "a", "a", "a"]
```

The `.includes` method checks the entire string for the provided text, note the check is case sensitive!

```
const truth = "JavaScript is a really fun language!";

console.log(truth.includes('fun')); //true
console.log(truth.includes('a')); //true
console.log(truth.includes('go')); //false
```

Just like `.startsWith` and `.endsWith` the `.includes` method will return `true` or `false`.

Start position

Unlike the other methods, `.includes` takes a second optional position parameter. This is used to tell `.includes` about where it should start checking in the string for the value.

```
const truth = "JavaScript is a really fun language!";

console.log(truth.includes('J')); //true
console.log(truth.includes('J', 10)); //false
```

We will see this again when we get to the `.includes` method in the

ES7(ES2016) & Beyond chapter.

`.repeat`

One more method to look at is the `.repeat` method is pretty straight forward, it allows us to repeat a string a given number of times.

```
const nan = 'NaN'.repeat(6);
console.log(nan + ' BATMAN!'); //"NaNNaNNaNNaNNaNNaN BATMAN!"
```

There are a few exceptions to what you can pass in as the count. It can not be a negative number and if it is a decimal number it will be rounded to an integer.

Unicode

New in ES6 for Unicode is the ability to represent Unicode as code points. Previously this was not possible because you could only represent a unicode character with up to 4 hexadecimal digits. So any Unicode character that required more that 4 you needed to create what is called a surrogate pair.

However in ES6 we can use the `\u{}` syntax to include up to 6 digits, enough to represent all the Unicode characters. It is pretty straight forward.

```
console.log('\u{1F44D}'); //👍
```

Regex

Regex in ES6 also got a few additions. There are two new flags available to use, the `y` or sticky flag, and the `u` flag. The `u` flag is used for unicode characters. For example, say we have a really cool bit of text.

```
"Man I really love 🚀, they are the best!"
```

And we wanted to see if there was the rocket ship emoji in there. Well we

And we wanted to see if there was the rocket ship emoji in there. Well we could do something like this

```
console.log("Man I really love 🚀, they are the best!".match(/\u{1F680}/u))
//[ '🚀',
//index: 18,
//input: 'Man I really love 🚀, they are the best!' ]
```

Note you could actually do this as well `.match(/🚀/)`, you will notice that we do not need to use the `u` flag here. The `u` flag is used when we pass a unicode code point `\u{}` to our expression. We can also do a range, so assume we wanted to get all the emojis used in a bit of text.

```
"Wow I really love this new phone I got 😃! Although the battery is not as good as my old on
```

Using the `u` flag we could match on a range of characters.

```
console.log("Wow I really love this new phone I got 😃! Although the battery is not as good
//["😃", "😟"]
```

This will match globally, `g`, all the faces from 😀 to 😂. And again we could just use the emjoi if we wanted `.match(/[😀-😂]/)`

Sticky match

The last thing I wanted to talk about with regex is the `y` flag, this is the sticky flag. It is used to allow us the chance to determine where to start our search. Before we dive into it, we need to look at the `lastIndex` property.

```
let regex = /really/g;
let text = "I really love pizza, is there really a better food?";
console.log(regex.exec(text)); // [ 'really', index: 2, input: 'I really love pizza, is there

console.log(regex.lastIndex); //8

console.log(regex.exec(text)); //[ 'really', index: 30, input: 'I really love pizza, is there

console.log(regex.lastIndex); //36
```

When you use the `g` flag and run an `exec` it will find the first match and set the `lastIndex` property. The next time you run the `exec` method it will use that index and start from there. With the `y` or sticky flag, we can set the `lastIndex` to let RegEx know where to start looking.

```
let regex = /really/y;
let text = "I really love pizza, is there really a better food?";
console.log(regex.exec(text)); //null
regex.lastIndex = 30;

console.log(regex.exec(text));
//[ 'really',
// index: 30,
// input: 'I really love pizza, is there really a better food?' ]
```

This can be helpful if you need to check for a bit of text starting from a specific point in your code.

Additional Resources

- https://mathiasbynens.be/notes/es6-unicode-regex
- http://www.loganfranken.com/blog/831/es6-everyday-sticky-regex-matches/

let & const

WE'LL COVER THE FOLLOWING

- Scope
- let
  - block scope vs function scope
  - Temporal Dead Zone
- const
  - Not immutable.

In ES6 there are two new keywords available that allow us to define variables: `let` and `const`.

Before ES6, if you wanted to create a variable you would use the `var` keyword, and along with this keyword you would also define a name.

```
var author = "Ryan Christiani";
```

We use the equals sign to assign the value to the variable. To use the variable and the value stored inside, we simply need to write `author`. This would refer to the value `"Ryan Christiani"`.

You can create a variable without the use of the `var` keyword, but this is discouraged as it creates an undeclared global variable. But, what do we mean when we say a global variable?

Scope

In programming languages we have this idea of scope: when you define a variable it is scoped to a specific section of code. Let's look at some code and break it down:

```
var rocket = "Falcon9";

function launch() {
    console.log('Launching ' + rocket);
}

launch(); // Launching Falcon9
```

In the example above, we have created a variable called `rocket` and it has been created outside of our `launch` function. Because it was created outside of the function it will be accessible inside of it via a closure. When a function is defined in JavaScript it will capture the variables that were available to it and make them available inside of the function.

So when we run `launch();` we see the text `Launching Falcon9`. If we changed the definition of the `rocket` variable so that is it is defined inside of a function called `rocketName()`, our `launch()` function will not work.

```
function rocketName() {
    var rocket = "Flacon9";
}

function launch() {
    console.log('Launching ' + rocket);
}

launch(); // Launching undefined
```

Variables created with the `var` keyword are **function scoped**, so unless they are created outside of a function, you will have no access to them. The code above will not work, because `rocket` exists only in the `rocketName` function.

let

In ES6 one of the new keywords is `let`, and its usage is just like using `var`.

```
let author = "Ryan Christiani";
```

When you refer to a variable created with `let` you do it just the same as you would with `var`. Referencing `author` would get you the value of `Ryan Christiani`.

The one key difference here is that a variable defined with `let` is a **block scoped** variable. Remember when we said that a `var` was a **function scoped** variable? Let's look at the difference between block and function scope.

block scope vs function scope

When we looked at `var` we saw that if we defined a variable inside of a function, it created the variable in there, and it is available inside of that function but not outside of it.

```
function setName() {
    var name = "Ryan";
}

console.log(name); //Undefined
```

If we declared a variable using `var` inside of a block statement, or anything with `{}` (like a conditional for example), it will make that variable available outside of that block!

```
if(true) {
    var name = "Ryan";
}

console.log(name); //Ryan
```

And if you are familiar with other languages, say C for example, you might think this to be odd, since C has the ability to create block scope. Enter `let`, in ES6 `let` allows us to create a block scoped variable.

```
{
```

```
    let name = "Ryan";
}


console.log(name); //undefined

if(true) {
    let name = "Ryan";
}

console.log(name); //undefined
```

Variables defined with `let` will behave the same inside of a function as a variable defined with `var` would. However, now we have the benefit of block scope.

One place this works particularly well is when we are creating a variable to use in a `for` loop. Since `var` is function scoped it does not create a value that stays inside the loop.

```
for(var i = 0; i < 10; i++) {
    console.log(i);
}

console.log(i); //10
```

This can be a little confusing. If we change the variable declaration from `var` to `let`, however, it will change the behavior.

```
for(let i = 0; i < 10; i++) {
    console.log(i);
}

console.log(i); //ReferenceError: i is not defined
```

Temporal Dead Zone

There is one more gotcha that I want to point out, and this is something called the Temporal Dead Zone. In JavaScript, when the browser interprets your

code it will do a pass where it looks for any declarations. However, it will not

assign the value just yet. Because of this it is possible to use a variable before it has a value: it will simply print `undefined`.

```
console.log(person); //undefined
var person = "Ryan";
```

The browser knows there is a variable called `person`, it just doesn't know what the value is. With the new `let` keyword, this is no longer the case.

```
console.log(person); //ReferenceError
let person = 'Ryan';
```

const

The `const` keyword behaves like `let`. It is block scope, but there is one difference that is pretty important. When you declare a variable with `const` it creates a read-only value, meaning you can use the value stored in it, but you can not reassign the value.

Before `const`, a common convention was to use uppercase names to hint that the value stored in this variable was a constant value.

```
var API_KEY = '195fhbgBqg268334mf124';
```

The obvious problem with this is that it COULD be reassigned whenever you wanted. With `const` this is not possible.

```
const api_key = '195fhbgBqg268334mf124';
api_key = 'some other key'; //TypeError: Assignment to constant variable
```

Trying to reassign the value throws a TypeError, which is great. If you defined a variable to be constant, you want to make sure it is just that.

Not immutable.

There is another small gotcha to watch out for: assigning a value here does not make it immutable. We are able to assign objects to `const` variables.

```
const person = {
    name: 'Ryan Christiani'
}
```

And since a `const` variable is a read-only value, you might assume that altering the object would throw an error, however this is not the case.

```
const person = {
    name: 'Ryan Christiani'
}
person.age = 31;

console.log(person);
```

This is completely valid; since the object's keys are not protected here you are able to alter and add to them. The value stored on `person` is still an object. If you are looking for immutability, consider using `Object.freeze`. Introduced in ES5.1, this method will freeze and prevent new properties from being added.

```
const person = {
    name: 'Ryan Christiani'
}
Object.freeze(person);

// Try changing a field in frozen object
person.name = 'Instructor Ryan Christiani'
// Try adding something to a frozen object
person.age = 31;

// frozen object doesn't change
console.log(person);
```

Template Literals

WE'LL COVER THE FOLLOWING

- Template Literals
  - Multi-line
  - Expressions
  - HTML Templates
  - Tagged Templates
    - Reusable Templates

Template Literals

Introduced in ES6 is a new way in which we can create a string, and that is the Template Literal. With it comes new features that allow us more control over dynamic strings in our programs. Gone will be the days of long string concatenation!

To create a template literal, instead of `'` or `"` quotes we use the `` ` `` character. This will produce a new string, and we can use it in any way we want.

```
let newString = `A string`;
console.log(newString);
```

Multi-line

The great thing about Template Literals is that we can now create multi-line strings! In the past, if we wanted a string to be on multiple lines, we had to use the `\n` or new line character.

```
let myMultiString = 'Some text that I want\nOn two lines!';

console.log(myMultiString);
```

With a Template Literal string, we can just go ahead and add the new line into the string as we write it.

```
let myMultiString= `This will be
on two lines!`;

console.log(myMultiString);
```

This will produce a string with a new line in it. The ability to do this with expressions makes Template Literals a really nice templating language for building of bits of HTML that we will cover later. But what about concatenation? Let's look at how we can dynamically add values into our new Template Literals.

Expressions

In the new Template Literal syntax we have what are called expressions, and they look like this: `${expression}`. Consider the code below.

```
let name = `Ryan`;

console.log(`Hi my name is ${name}`);
```

The `${}` syntax allows us to put an expression in it and it will produce the value, which in our case above is just a variable that holds a string! There is something to note here: if you wanted to add in values, like above, you do not need to use a Template Literal for the `name` variable. It could just be a regular string.

string.

```
console.log(`Hi my name is ${'Ryan'}`);
```

This will produce the same output. These expressions do more than just let us put in variables that contain strings in them. We can evaluate any sort of expressions that we would like.

```
let price = 19.99;
let tax = 1.13;

let total = `The total prices is ${price * tax}`;
console.log(total);
```

We can also use this with a more complex object.

```
let person = {
    firstName: `Ryan`,
    lastName: `Christiani`,
    sayName() {
        return `Hi my name is ${this.firstName} ${this.lastName}`;
    }
};

console.log(person.sayName());
```

Here we have a `person` object with a `sayName()` method on it: this uses the shorthand method definition we looked at in the Objects chapter! We can access the properties from an object inside of the `${}` syntax.

HTML Templates

With the ability to have multi-line strings and use Template Expressions to add content into our string, this makes it really nice to use for HMTL templates in our code.

Lets imagine that we get some data from an API that looks something like this:

```
{
    "id": 1,
    "name": "Bulbasaur",
    "base_experience": 64,
    "height": 7,
    "is_default": true,
    "order": 1,
    "weight": 69,
    ...
}
```

This “imaginary” API is of course the pokeapi! With this data structure in mind, let’s create the markup that would show this Pokemon.

```
function createMarkup(data) {
    return `
        <article class="pokemon">
            <h3>${data.name}</h3>
            <p>The Pokemon ${data.name} has a base experience of ${data.base_experience}, the
        </article>
    `
}
```

Without having to use a library like Handlebars or Mustache we can create nice and easy-to-use templates in our JavaScript!

Let’s look at our Pokeman example in action (*note: we are using jQuery to modify the DOM. You can ignore this example if you don’t understand jQuery*)

HTML JavaScript

```javascript
let data = {
    "id": 1,
    "name": "Bulbasaur",
    "base_experience": 64,
    "height": 7,
    "is_default": true,
    "order": 1,
    "weight": 69
};

function createMarkup(data) {
    return `
        <article class="pokemon">
            <h3>${data.name}</h3>
            <p>The Pokemon ${data.name} has a base experience of ${data.base_experience}
        </article>
    `
}

$(document).ready(function () {
$('#content').html(createMarkup(data));
```

```
22 });
23
```

output

Bulbasaur

The Pokemon Bulbasaur has a base experience of 64, they also weigh 69

Tagged Templates

Another features of Template Literals is the ability to create Tagged Template Literals. The way this works is that you create a function and this function will look like any other function, however when you execute it, that is when it looks different. Consider the function below.

```
function myTaggedLiteral(strings) {
    console.log(strings);
}

myTaggedLiteral`test`; //["test"]
```

First off, you will notice there are no `()` when we call the function! We apply a Template Literal where the parentheses would be. As a parameter to our function we get an array of the strings in our literal. Let's expand on the string we send to the function and we will have it include an expression, and we will include a new parameter in our function as well.

```
function myTaggedLiteral(strings,value) {
    console.log(strings,value);
}
let someText = 'Neat';
myTaggedLiteral`test ${someText}`;
//["test", ""]
// "Neat"
```

When we use an expression we can access that from the next parameters and this keeps going. Say we added another expression.

```
function myTaggedLiteral(strings,value,value2) {
    console.log(strings,value, value2);
}
let someText = 'Neat';
myTaggedLiteral`test ${someText} ${2 + 3}`;
//["test", ""]
// "Neat"
// 5
```

This is pretty powerful: it allows you to take the data used in a string and manipulate it to your liking.

Reusable Templates

Let's look at a simple use case for this. If you remember from above, we saw how Template Literals work really great for, well, making templates! Let's take that a step further and create a function that would allow us to create reusable templates. The idea here is that we can create the initial template and then pass in data for it to use later.

```
const student = {
    name: "Ryan Christiani",
    blogUrl: "http://ryanchristiani.com"
}

const studentTemplate = templater`<article>
    <h3>${'name'} is a student at HackerYou</h3>
    <p>You can find their work at ${'blogUrl'}.</p>

</article>`;

const myTemplate = studentTemplate(student);
console.log(myTemplate);
//Output will look like this!
//<article>
//    <h3>Ryan Christiani is a student at HackerYou</h3>
//    <p>You can find their work at http://ryanchristiani.com.</p>
//</article>
```

Let's look at implementing our `templater` function.

```
const templater = function(strings,...keys) {

}
```

The first thing you will notice is this `...keys` parameter. The `...` syntax is what's called Rest Parameters; it basically will gather any parameters the function has and create an array for us. We will dig deeper into that in the Spread Operator & Rest Parameters chapter.

The next thing we want to do is return a function that is going to access our object. The returning of the function is what allows us to call and pass in our student data, like this: `studentTemplate(student)`.

```
const templater = function(strings,...keys) {
    return function(data) {

    }
}
```

With this data now available to us we need to perform some manipulation. The process is as follows. First, we need to create a copy of the `strings` array. We make a copy in case we want to reference the original later. Then we need to loop through the array of `keys`, and for each one of those, grab the data from the object that matches the key (notice how in this example we pass in a string in the `${}`) and place it in our array where needed. Finally, we need to join it all back together as a string and return it from the function!

```
function templater(strings, ...keys) {
    return function(data) {
        let temp = strings.slice();
        keys.forEach((key, i) => {
            temp[i] = temp[i] + data[key];
        });
        return temp.join('');
    }
};
```

Let's see our example in action:

```
function templater(strings, ...keys) {
    return function(data) {
```

```
        let temp = strings.slice();
        keys.forEach((key, i) => {
            temp[i] = temp[i] + data[key];

        });
        return temp.join('');
    }
};

const student = {
    name: "Ryan Christiani",
    blogUrl: "http://ryanchristiani.com"
}

const studentTemplate = templater`<article>
    <h3>${'name'} is a student at HackerYou</h3>
    <p>You can find their work at ${'blogUrl'}.</p>

</article>`;

const myTemplate = studentTemplate(student);
console.log(myTemplate);
//Output will look like this!
//<article>
//    <h3>Ryan Christiani is a student at HackerYou</h3>
//    <p>You can find their work at http://ryanchristiani.com.</p>
//</article>
```

You will notice that this is not an exhaustive example. We have no way to accommodate nested data or array values; it is simply just strings. But I hope this example helps to illustrate what you can start doing with Tagged Template Literals.

Objects

WE'LL COVER THE FOLLOWING

- Shorthand methods
- Shorthand properties
- Computed properties
- Object.assign()

Objects have received some new updates in this edition of the language, including everything from new methods to updated syntax.

Shorthand methods

One nice addition to objects is the shorthand method syntax. We will see this pattern come up again when we talk about classes. As a reminder, here is what the original syntax looks like.

```
let flower = {
    height: 10,
    colour: 'yellow',
    grow: function() {
        this.height += 5;
    }
}
```

Here we have a simple `flower` object which has a `height` and `colour` and a `grow` method. When defining a method we provide a function as a value to the property on an object. In this case we use an anonymous function. We could, if we wanted, also provide a named function.

```
let flower = {
    height: 10,
    colour: 'yellow',
    grow: function growMethod() {
```

```
        this.height += 5;
    }
}
```

The benefit of providing a named function is that we could use this name inside of our method if we need to call it recursively. With the new shorthand method syntax, we can exclude the anonymous function all together.

```
let flower = {
    height: 10,
    colour: 'yellow',
    grow() {
        this.height += 5;
    }
}
```

Look at that! I think this looks really nice.

Shorthand properties

Another new feature is the ability to create object properties via a shorthand syntax. There is not much to this, and we will see this being used later in the Destructuring chapter. Let's consider the code below: assume we have some variables that we want to add to an object.

```
let firstName = "Ryan";
let lastName = "Christiani";

let person = { firstName, lastName };
```

The keys in our new object will be the names of the variables we provided it. The above is a shorthand for doing this:

```
let person = {
    firstName: firstName,
    lastName: lastName
};
```

It really is that simple; this new syntax allows us to create objects from existing variables with ease. A place this pattern is kind of nice is the revealing module pattern.

The idea of the revealing module pattern is that you have a function, that is acting as a module, that has data you want to keep private. From the module

acting as a module, that has data you want to keep private. From the module you return a public object that only reveals what you want to the user.

```
function slider() {
    let currentPosition = 0;
    function updatePosition(newPosition) {
        currentPosition = currentPostion + newPosition;
    }
    function resetPosition() {
        currentPosition = 0;
    }
    return {
        updatePostion,
        resetPostion
    };
}
```

Above we have a little slider module that doesn't actually do anything, but it illustrates the point. We have some private data, the `currentPosition` value, we also have two functions that will mutate that value. We expose or reveal the functions that we want, and with the shorthand property method we can make this nice and easy!

Computed properties

Computed properties is another new thing we can do with object properties in ES6. When creating an object, properties are made up of key value pairs. Typically a key could be a name like `firstName` or it could be wrapped in quotes `"firstName"`. With computed properties we are able to use expressions or existing variables to create keys.

```
let keyName = 'firstName';
let person = {
    [keyName]: 'Ryan'
};
```

As mentioned above, we can use expressions to create keys.

```
let person = {
    ['first' + 'name']: 'Ryan'
};
```

Object.assign()

The last thing involving objects that I want to go over is the `.assign()` method. This new method objects allows us to create copies of existing objects

method. This new method objects allows us to create copies of existing objects, and also mix objects together.

The method is fairly straightforward. The first argument in the method is the target you want to assign, and any arguments after that are sources that you want to add.

As an example, let's think about a video game. We have some base objects, like a creature, and we have different versions of these creatures.

```
let bat = {
    weight: 10,
    strength: 4,
    altitude: 0,
    fly(newAltitude) {
        this.altitude += newAltitdude;
    }
};
```

We have our base object that has some simple stats, now say we have a greater bat!

```
let greaterBat = {
    weight: 15,
    strength: 7
};
```

The new bat has a few newer properties, but we also want it to be able to fly! There is a way to solve this, we could use inheritance via function prototypes, or we could use classes. We will look at classes later. For this example, we will favour composition of our object. Let's change `greaterBat` to `greaterBatProps` so we can use it in `Object.assign()`.

```
let greaterBatProps = {
    weight: 15,
    strength: 7
};

let greaterBat = Object.assign({}, bat, greaterBatProps);
console.log(greaterBat);
```

Using `Object.assign()` we are composing greaterBat into bat and finally into

an empty object that acts as our target. The order of our arguments matter here, if we had called it like `Object.assign({},greaterBatProps,bat);` we would have a whole object. However because `bat` came last in this example, it would override anything in `greaterBatProps`. When you look at this read it from the right to the left as this is how it will compose!

This pattern is really great, and there is another great benefit to `Object.assign()` and that is to use it to copy objects! Consider the example below.

```
let person = {
    name: 'Ryan'
};

let person2 = person;

person2.name = "Erin";

console.log(person2.name); //Erin
console.log(person.name); //Erin
```

We have created a `person` object, and gave it a simple property. Then we create a new variable called `person2` and assign it `person`. You might think that this would add a copy of the person object, but a new person object.

However that is not the case, it just uses `person` as a reference, so changing the `name` property on `person2` is going to change it on `person`!

Using `Object.assign()` we can create a copy of `person` with ease!

```
let person = {
    name: 'Ryan'
};

let person2 = Object.assign({},person);

person2.name = "Erin";

console.log(person2.name); //Erin
console.log(person.name); //Ryan
```

Remember that the first argument to `Object.assign()` is the target, so using a blank object, we are creating a copy of the `person`.

Arrays

WE'LL COVER THE FOLLOWING

- for...of
- Typed Arrays
- Array.from()
- Array.of()
- find() & findIndex()
- Values, Keys & Entries
  - values()
  - keys()
  - entries()
- copyWithin()
  - Why use copyWithin?

Arrays in ES6 have also gotten a little love. There are quite a few new methods available for use on them, and there is also a brand new way to iterate over arrays. In JavaScript there is something called Iterator Objects, these are things like arrays, strings and some things we haven't looked at yet called Map and Set. In ES6 the `for...of` loop is a new construct that allows us to iterate over these objects.

for...of

The syntax for the `for...of` loop is very similar to that of the `for...in` loop, but instead of looping through the keys of an object it will loop through the elements of the array.

```
let programmingLangauges = ['JavaScript','Ruby','GO'];
```

```
for (let language of programmingLangauges) {
    console.log('I really like ' + language);
}

//"I really like JavaScript"
//"I really like Ruby"
//"I really like GO"
```

There is not much else to look at here. The real benefit is that we don't have to create a `for` loop the old-fashioned way.

```
let programmingLangauges = ['JavaScript','Ruby','GO'];

for (let i = 0; i < programmingLangauges.length; i++) {
    console.log('I really like ' + programmingLangauges[i]);
}
```

This does the same thing as the above, but it is a little more verbose, and we have to configure the loop. If we simply just want to loop over an array, the `for...of` loop is great for that!

Typed Arrays

Something I wanted to briefly look over are Typed Arrays. In ES6 you now have the ability to create arrays that are made up of just elements of a specific data type. JavaScript is a dynamically-typed language, meaning that when we define a variable we don't have to define that it will be a string or number. The reason for creating Typed Arrays is to create data that will perform better than the average array. This is typically for data used in WebGL programs or something that has to deal with binary data.

There are a few ways to create a Typed Array.

```
let newTypedArray = new Int8Array(5);
newTypedArray[0] = 1;
newTypedArray[1] = 2;
newTypedArray[2] = 3;
```

You can do it by using the `new` keyword and using one of the available types

`Int8Array`, `Int16Array`, `Int32Array`, etc. You also have to the length that would like your array to be in the constructor. To find out more check out the MDN article.

Another way that we can create a Typed Array is by passing the values directly into the array type that you want to create.

```
let newTypedArray = new Int8Array([1,2,3]);
```

This will do that same as the above. But, what is the point of using a typed array? When an interpreter know that the values it will encounter will never change from a specific type, it can do some optimizations for that! And this makes things fast.

Array.from()

The `Array.from` method allows us to take an array-like object or Iterator Object and return an actual array from this. Let's look at some examples!

The most useful example is when working with the `arguments` objects. This is a value that we have access to when we call a function (not an arrow function though).

```
function add() {
    console.log(arguments);
}
add(100, 200, 300); //{ '0': 100, '1': 200, '2': 300 }
```

The problem here is that it's not an actual array (we will solve this problem another way later in the book). We can, however, use `Array.from` to turn this into an array. The real issue here is that we can't use an array method like `.reduce()` to sum up these numbers.

```
function add() {
    args = Array.from(arguments);
    return args.reduce(function(num1,num2) { return num1 + num2 }, 0);
}
```

```
let result = add(100,200,300);
console.log(result); //600
```

We can also do this with stuff like strings. Strings are what we call an Iterator Object.

```
let arrayFromString = Array.from('This is neat!');
console.log(arrayFromString); //["T", "h", "i", "s", " ", "i", "s", " ", "n", "e", "a", "t",
```

`Array.from` will also accept another argument, a map function used to map over the provided array-like/iterable object.

```
function double() {
    return Array.from(arguments,function(n) { return n * 2 });
}
console.log(double(1,2,3)); //[2,4,6];
```

Array.of()

The `Array.of` method is a simple method that allows us to create an array of values. The syntax looks like this:

```
const numbersArray = Array.of(1,2,3,4,5,6);
```

It creates an array OF the values passed to it. There is not much else to it!

find() & findIndex()

If you have ever used a library like Underscore or Lodash, you have probably used a method like `_.find()` that will allow you to search through an array and find the first instance of a value. Let's look at an example and assume we have a set of data like this:

```
const students = [
    {
        name: 'Steve',
        course: 'History'
    },
    {
        name: 'Mary',
        course: 'Science'
    },
    {
        name: 'Lisa',
        course: 'Physics'
    },
    {
        name: 'Michelle',
        course: 'Physics'
    }
];
```

In this array of student objects, perhaps we want to find a student that is taking Physics. Using `Array.find` we can find the student like this.

```
const physicsStudent = students.find(function(student) {
    return student.course === 'Physics';
});

console.log(physicsStudent);
/*
{
    course: 'Physics',
    name: 'Lisa'
}
*/
```

The `.find` method will find the first instance of your search: you provide the method with a callback function that is passed each element from the array. The callback returns a check, in this case what we want to be true, and only once the condition `student.course === 'Physics'` is true will the `.find` method stop and return that element from the array.

It's important to remember that this will just find ONE instance of the search. If you want to get multiples, consider the `.filter` method.

The `.findIndex` method is pretty much identical, however instead of it returning the element that matches the check, it will return the index of that element in the array.

```
const physicsStudent = students.findIndex(function(student) {
    return student.course === 'Physics';
});

console.log(physicsStudent); //2
```

`.find` and `.findIndex` also accept an optional second argument: an object that should be used as the `this` in the callback if needed.

Values, Keys & Entries

These next three methods are similar to each other, so it makes sense to talk about them together. Each of these methods will return a new Iterator Object; we will explore these a little more when we get to the `Map` and `Set` chapters. But, an Iterator Object is an object that implements a `.next` method. This is a method we can use to iterate through values!

values()

The `.values` method will take an array and return an object that will allow us to step through each value individually. Let's keep using the `students` object from above for our examples here.

```
const studentValues = students.values();

console.log(studentValues.next().value);
/*
{
    name: 'Steve',
    course: 'History'
}
*/
console.log(studentValues.next().value);
/*
{
    name: 'Mary',
    course: 'Science'
}
*/
```

The `.next` method will let you iterate, one at a time, through the values in the

array! We could also use this in a `for...of` loop if we wanted, however that would just be very similar to passing an array to the `for...of` loop.

keys()

Similar to `.values`, the `.keys` method will return an Iterator Object, but this time the values returned are the index number for the current iteration you are on.

```
const studentKeys = students.keys();

console.log(studentKeys.next().value); //0
console.log(studentKeys.next().value); //1
console.log(studentKeys.next().value); //2
```

Again we could pass this to a `for...of` if needed. One thing to note is that when you exhaust the next available iteration of an these Iterator Objects, it will return `undefined` as a value.

> sentence needs clarification

entries()

Last is the `.entries` method which is very similar to the first two. However, the values returned from this method are in an array. From this array the first element is the index of the element and the second is its value.

```
const studentEntries = students.entries();

console.log(studentEntries.next().value);
/*
[0, {name: 'Steve',course: 'History'}]
*/
console.log(studentEntries.next().value);
/*
[1, {name: 'Mary',course: 'Science'}]
*/
```

The `.values`, `.keys` and `.entries` methods all return Iterator Objects that

allow us to step through the values, each with a different version of the data. We will see more Iterator Objects in chapters to come!

copyWithin()

The `.copyWithin` method provides a way for you to take a portion of your array and copy it to a new location in the same array. It takes three arguments, the target index to copy too, the start index, and the end index.

An example will help explain it best:

```
const nameArray = ['R','Y','A','N'];

console.log(nameArray.copyWithin(0,2,3)); //['A','Y','A','N'];
```

In the example above, we have an array of letters, using the `.copyWithin` method I can say this. I want to target the first index(`0`) then start at the third index(`2`) and copy up to but not including the fourth index (`3`). Then place that in the target location `0`. If I wanted to have the first letter be `N` in the array I could change the `.copyWithin` call to look like this.

```
const nameArray = ['R','Y','A','N'];

console.log(nameArray.copyWithin(0,3,4));
```

It is important to note that since `4` is not an index in the array, it will just go up until the end of the array's length. You may also use negative numbers here as well.

```
const nameArray = ['N','Y','A','N'];

console.log(nameArray.copyWithin(-1,1,2));//["N", "Y", "A", "Y"]
```

The `-1` here represents the the `'N'` or last element in the array, the negative

number will just loop back around. In this case it will place the `Y` at the end.

Why use copyWithin?

The `.copyWithin` method is a method that is mostly going to be used for high performance systems, like games or programs written for a lower level. Since this is something we can also use with Typed Array's it will help save memory since we are not creating new space in memory, we are simply moving it.

Destructuring

Destructuring is a way for us to extract data from arrays and objects without having to manually create variables. Take this object for example.

```
let person = {
    name: "Ryan",
    age: 30,
    location: "Toronto"
};
```

An example of the destructuring syntax would be this.

```
let { age : personAge } = person;
console.log(personAge);
```

In the destructuring expression the left-hand value is the key from the object, and the right is the new variable we would like to create. Don't want to create a new variable? We can use a special shorthand to just create a new variable called `age`.

> this paragraph is confusing

```
let { age } = person;
console.log(age);
```

The great thing about this is that we can do this on multiple keys.

```
let { age , location: currentLocation} = person;
```

Note that we mix and match the shorthand with the longer assignment. If we had done this with just the shorthand `location` key as the variable, there would be an issue! Since `location` refers to the `window.location` property, changing that will change the location of the page! So in this case we have to assign a new variable name.

We can even use computed properties (like we saw in the Objects chapter) to pull out the values.

```
let key = 'age';

let { [key]: keyAge } = person;
```

In this case we have a string value stored on our variable, and this will now act as our key. Along with objects we can also destructure arrays. Instead of being able to extract the values based on the keys from an object, we can just use variables that represent elements in the array.

```
let numbers = [1,2,3,4,5];

let [first,second] = numbers;
console.log(first,second); //1 2
```

Unlike destructuring an object, we can't just use the key name to select any element in the array. Luckily we have the ability to skip elements in the array.

```
let numbers = [1,2,3,4,5];

let [first,second,,fourth] = numbers;
console.log(first,second,fourth); // 1 2 4
```

Destructuring is a pretty common practice when pulling from a module, and we will see this a little later on. But if you have done any React work with React-Router you will have probably seen code like this (look at the highlighted code).

```
import React from 'react';
import ReactDom from 'react-dom';
import { Router, Route, Link } from 'react-router'

let App = React.createClass({
    render() {
        return (
            <div>
                <h1>My App</h1>
                <Link to="/somepage">A Link</Link>
            </div>
        );
    }
});
```

This is just a look at how we can use destructuring and modules together, more to come later!

Default Parameters

Default Parameters are a great addition to the language. They allow you to, well, add default parameters. And this solves a very simple problem.

```
function getData(url) {
  url = url || 'http://api.data.com';
  return fetch(url)
    .then(res => res.json);
}
```

Consider the above, a simple function to get data from a source. If a URL is passed in, it will go and get the data from that URL. However if no URL is provided, it will get the data from `http://api.data.com`, a default source.

With the new ES6 Default Params we are able to add that to our function definition and make it more apparent what is happening.

```
function getData(url = 'http://api.data.com') {
  return fetch(url)
    .then(res => res.json);
}
```

This really cleans things up. With the ability to create Default Params, we can have more flexible and reliable functions.

The way Default Params works is that if nothing is passed in it will take the value stored in the function definition, in this case `url = 'http://api.data.com'`. If you do pass in a value for that position it will override the section.

When working with Default Params you are able to mix and match, allowing you to create easy-to-understand optional parameters for your functions.

```
function dataHandler(method, url = 'http://api.data.com') {
  return fetch(url)
```

```
  return fetch(uri)
    .then(res => res.json);
}

dataHandler('POST');
dataHandler('GET');
```

The Spread Operator and Rest Parameters

WE'LL COVER THE FOLLOWING

- Rest Parameters
  - call()
- Spread Operator

Two of my favorite new features in ES6 are the Spread Operator and Rest Parameters. These two features really allow us to create powerful operations that might have required more work, and added more confusion than needed. Let's first take a look at Rest Parameters.

Rest Parameters

From time to time you might want to write a function that would take an unknown number of arguments. JavaScript has the ability to do this through the `arguments` keyword. Take a `sum()` function, say it takes only two numbers

```
function sum(a,b) {
  return a + b;
}
```

Maybe we want this `sum()` to take any number of arguments? We can tell how many arguments have been passed to a function by using the `arguments` keyword.

```
function sum() {
  console.log(arguments);
}

let total = sum(1,3,4,6); //[1,3,4,6]
```

This is great: it looks like it returns an array for us to work with. We can go ahead and use the `reduce` method to reduce the numbers.

Reduce is a method that accepts a function and runs it for each value in an array. It can be used for much more than just getting the sum of some values, but in our case we will use it just for that!

```
const sum = function() {
  return arguments.reduce( (pre,cur) => {
    return pre + cur;
  });
};

const total = sum(1,2,3,4); //"TypeError: arguments.reduce is not a function
```

There is a small problem, however, as you will see if you run this code. The problem is that `arguments` is not an array. It is an array-LIKE object, meaning that it does not have all the array methods on it. So, how can we solve this problem? We can use the `.call()` method.

call()

`.call()` is a method that allows us to call a function with a provided value for `this`. As a quick example, imagine a simple object that has a `fullName` property.

```
const person = {
  fullName: 'Ryan Christiani'
}
```

Let's also assume we have a function like this:

```
function sayName() {
  console.log(this.fullName);
}
```

In the function above, `this` will be bound to the window. Using the `.call()` method we can call the `sayName` function and set the `this` context to be our

`person` object!

```
sayName.call(person); //Ryan Christiani
```

With this in mind we can use `Array.prototype.reduce`, the method that all Array's get, and use the `.call()` method on it passing `arguments` to get it to work.

Although this works, and it can be very powerful, it is not great as a beginner, and can be confusing to understand. Rest Parameters allow us to define a function that can accept any number of arguments, with a more straight forward syntax.

```
const sum = function(...numbers) {
    console.log(numbers);
};

function sayName() {
  console.log(this.fullName);
}

sum(1,2,3,4,5,6); //[1,2,3,4,5,6]
```

The `...` is the special prefix that both Rest Parameters and the Spread Operator use, as we will see later. The name following the `...` is the parameter name. The great thing about this is that the `numbers` parameter value is now an actual array, and we can call the reduce method on it!

```
const sum = function(...args) {
    return args.reduce(function(prev,curr) {
        return prev + curr
    });
};

console.log(sum(1, 2, 3, 4, 5));
```

There is something that I want to point out: you can use Rest Parameters to just gather the rest of the arguments from a function. Let's write a function that will take a value as the first argument and then multiply the rest of the values from there.

```
let multiply = function(mul,...args) {
  return args.map(function(num) {
    return mul * num;
  });
};

console.log(multiply(3,14,45,6,7,8) );//[42, 135, 18, 21, 24]
```

Spread Operator

As mentioned previously the syntax for the Spread Operator is similar to Rest Parameters: it uses `...` as well. It differs from Rest Parameters, however, in that you need to start with an array value and then you spread it out.

```
const numbers = [4,5,6,7];
console.log(...numbers); // 4 5 6 7
```

Let's look at a good example of this. Consider `Math.max()`, this method takes any number of arguments and it will return the maximum of them.

```
const max = Math.max(1,23,45,6,7,8);
console.log(max); // 45
```

If you have ever tried passing an array to `Math.max` you will know that this does not work.

```
const numbers = [23,46,12,63,23];

console.log(Math.max(numbers)); // NaN
```

Similar to the `.call()` method, there is a method on functions called `.apply()`. This method calls a function and supplies a context for the `this` keyword, as well as an array that we want to use as the values for a function. With the same `numbers` array from before, we could do something like this.

```
const numbers = [23,46,12,63,23];
const total = Math.max.apply(null,numbers);
console.log(total); //63
```

With the Spread Operator we can do what `.apply()` did, but much more cleanly. I showed a student `.apply` recently in a similar situation and it was a little confusing for them. However when I explained spread to them it made more sense! It is basically a better `.apply()` for this situation.

```
const numbers = [23,46,12,63,23];

const total = Math.max(...numbers);

console.log(total);// 63
```

Going further, we can use the Spread Operator as way to concatenate arrays together.

```
let numbers = [2,34,5];
let newNumbers = [74,52,...numbers];

console.log(newNumbers);
// [74, 52, 2, 34, 5]
```

With the ability to take an array of values and spread them out, it really opens up a lot of possibilities for us.

Arrow Functions

WE'LL COVER THE FOLLOWING

- this
  - Binding an Arrow Function
- arguments
- Additional Resources

In ES6 there is a new function syntax. There are some nuances to the syntax that we will be going over.

Before we get into the syntax for an Arrow Function, let's look at where the inspiration came from. CoffeeScript has had Arrow Functions for a while, there is a talk from Brendan Eich and Jeremy Ashkenas [(1)] and a post from Brendan [(2)] where they talk about how developers helped invent the future of JS. CoffeeScript was one of those languages that helped push JavaScript forward.

The syntax in CoffeeScript looks like this:

```
add = (a,b) ->
    a + b
```

For ES6 they settled on this syntax:

```
const add = (a,b) => {
  return a + b;
};

console.log(add(5, 10));
```

From an initial look at the syntax, you might be thinking “So, no function keyword?”. That is not quite accurate. The Arrow function can take many different forms. You can write an Arrow Function like an anonymous function.

```
const add = function(a,b) {
  return a + b;
}

const add = (a,b) => {
  return a + b;
}
```

You call the functions the same way `add(2,3)`. If you have a single-line statement for your function, you can remove the parentheses and the `return` keyword to reduce everything to one line.

```
const add = (a,b) => a + b;
console.log(add(5, 10));
```

Whoa! In a single-statement Arrow Function like the one above, it is an implicit return as opposed to an explicit return. This makes the use of Arrow Functions in functional programming patterns really nice!

```
const numbers = [2,3,4,5,6];
const doubled = numbers.map((n) => n * 2);
console.log(doubled);
```

Since the `.map` method takes a callback function to perform on each element, and it is usually a very small operation, this works great. In fact if you have one parameter for your function you can even leave the parentheses off!

```
const numbers = [2,3,4,5,6];
const doubled = numbers.map(n => n * 2);
console.log(doubled);
```

That being said, if you want to have the parenthesis and curly brackets, you are totally able to do so (*but then you have to use the return statement*).

```
const numbers = [2,3,4,5,6];
const doubled = numbers.map((n) => { return n * 2; });
console.log(doubled);
```

this

There is one place that the Arrow Function may get a bit confusing, and that is when we talk about the `this` keyword. The subject of `this` has been widely discussed. Sometimes it is intuitive as to what value it should be, and sometimes it is not.

Let's look at an example. On an object where we have a method, `this` is bound to the object that the method is on.

```
const person = {
  name: 'Ryan',
  sayName() {
    console.log(`Hi I am ${this.name}`);
  }
};

person.sayName();
```

In the above example `this` references the `person` object. Changing the function from an anonymous function to an Arrow Function, however, will alter the value.

```
const person = {
  name: 'Ryan',
  sayName: () => {
```

```
    console.log( Hi I am ${this.name} );
  }
};


person.sayName();
```

If you call the method you will get `Hi I am undefined`, and in strict mode it would throw `"TypeError: Cannot read property 'name' of undefined`. With the Arrow Function the `this` keyword is bound to what is called the lexical scope, or how I like to think of it, the parent scope. In the case of the browser, this might be the `window` object. We can, however, use this to our advantage!

We can use this to our advantage. Consider the code below, we have an object with a method. The `.showHobbies` method simply logs the hobbies to the console, or so we would hope!

```
const person = {
    name: 'Ryan',
    hobbies: ['Robots','Games','Internet'],
    showHobbies: function() {
        this.hobbies.forEach(function(hobby){
            console.log(`${this.name} likes ${hobby}`);
        });
    }
};
person.showHobbies();
// undefined likes Robots
// undefined likes Games
// undefined likes Internet
```

This doesn't work they way we want. The `this` keyword inside of the `.forEach` here is bound to the `undefined`. In your browser (*without the strict mode*) `this` would be bound to window. We can actually pass another argument to `.forEach` for the `this` value, so we could rewrite it like:

```
const person = {
  name: 'Ryan',
  hobbies: ['Robots','Games','Internet'],

  showHobbies: function() {
    this.hobbies.forEach(function(hobby){
      console.log(`${this.name} likes ${hobby}`);
```

```
    },this);
  }
};

person.showHobbies();
```

Here we pass the current `this` as the value we want to use as the context for `this` in `.forEach`. There is also a common trick that looks something like this.

```
const person = {
  name: 'Ryan',
  hobbies: ['Robots','Games','Internet'],

  showHobbies: function() {
    var self = this;
    this.hobbies.forEach(function(hobby){
      console.log(`${self.name} likes ${hobby}`);
    });
  }
};

person.showHobbies();
```

But with Arrow Functions we can use it to grab the parent scope(or the lexical scope) so that we don't have to change much of how we write our code.

```
const person = {
  name: 'Ryan',
  hobbies: ['Robots','Games','Internet'],

  showHobbies: function() {
    this.hobbies.forEach(hobby => {
        console.log(`${this.name} likes ${hobby}`);
    });
  }
};

person.showHobbies();
```

The scope for `this` in the arrow function is the same as the parent method.

Binding an Arrow Function

It should be noted that you can not bind the `this` context of an Arrow Function. Unlike a regular function, the arrow's have a lexical scope, so the scope is decided based on the call location. This means we can't use methods like `.call()`, `.apply()` or `.bind()` to change it.

```
const person = {
  city: 'Toronto'
}

const sayName = () => {
  console.log(this.city);
}

sayName.call(person); //undefined
```

arguments

There is another nuance of the Arrow Function that I want to point out: inside of an Arrow Function there is no access to the `arguments` keyword. If you are unfamiliar with `arguments` it is a keyword that supplies an array-like object that contains all of the arguments passed to the function. For more information, refer back to the Spread Operator and Rest Parameters chapter.

Additional Resources

- 1:https://youtu.be/QTj6Q_zV1yg?list=PL37ZVnwpeshGoyAz6XzjKtlgp9XrjVry_
- 2:https://brendaneich.com/tag/javascript-ecmascript-harmony-coffeescript/

Classes

WE'LL COVER THE FOLLOWING

- Class emulation in JavaScript
  - Inheritance with functions
- Class
  - super()
  - Static Methods
- But I read I should never use a class...

Classes are now here in JavaScript, up until now we have had many ways to ways to emulate this concept. For awhile now we have been emulating classes with the use of functions. The introduction of classes to the languages was met with a lot of opinions, some positive, most negative. Classes have been an important part of many languages, as they allow you a means to create blueprints for common elements in your program.

Class emulation in JavaScript

In JavaScript we can use functions and objects to emulate classes. Functions can be used to create constructors that, when used with the `new` keyword, can instantiate new objects.

```
function Plane() {
        this.wings = 2;
        this.speed = 100;
        this.altitude = 0;
}
```

Convention has it that constructors are named with a capital first letter. To create a new `Plane` we use `new`.

```
const myPlane = new Plane();
```

If we want to add methods to the `Plane` we can use the `prototype` property to add them on.

```
Plane.prototype.fly = function() {
        this.altitude = 30000;
};
```

This will allow us to use the `fly` method on our new `myPlane`.

```
function Plane() {
        this.wings = 2;
        this.speed = 100;
        this.altitude = 0;
}

Plane.prototype.fly = function() {
        this.altitude = 30000;
  console.log('Flying. Altitude:', this.altitude, 'Speed:', this.speed);
};

const myPlane = new Plane();
myPlane.fly();
```

Inheritance with functions

It we want to create inheritance with functions, we can also use the `prototype` property to create an inheritance chain.

```
function JetFighter () {
        this.speed = 1000;
}

JetFighter.prototype = new Plane();

const jet = new JetFighter();
jet.fly();
```

We won't go much further into object-oriented JavaScript. There is an excellent book by Nicholas Zakas called *Principles of Object-Oriented JavaScript* that I recommend checking out. I should point out that the `class` keyword is actually just syntactic sugar for the method shown above.

Class

Let's now look at how the new `class` syntax works in ES6. If you come from a language like Java this might look very familiar.

To define a class we use the `class` keyword and give it a name; this is a class declaration.

```
class Human {}
```

We can also create a class on a variable; this is called a class expression.

```
const Human = class {}
```

By convention we use a capital letter for the class name. There is a a special method that needs to be included if we would like to apply initial properties to our class. This method is the `constructor()` method. You will notice we use method shorthand we saw earlier in the Objects chapter.

```
class Human {
        constructor() {
                this.height = 185;
        }
}
```

The `constructor()` method is called whenever we create a new instance of our class using the `new` keyword. Take a look at the example below.

```
const ryan = new Human();
console.log(ryan.height); //185
```

We can pass arguments into the `constructor` method.

```
class Human {
        constructor(height) {
                this.height = height;
        }
}

const ryan = new Human(190);
console.log(ryan.height);
```

Let's expand on our `Human` class and add some more properties. Maybe we are creating a video game, so we add `speed`, `strength`, and `location` properties. We also add a `walk` method. Something to note about the syntax for adding new methods to a class is that we do not add a `,` for each new method.

```
class Human {
        constructor() {
                this.height = 185;
                this.weight = 220;
                this.strength = 10;
                this.speed = 10;
                this.location = {
                        x: 0,
                        y: 0
                }
        }
        walk(x,y) {
                this.location.x += x;
                this.location.y += y;
    console.log('Walked to new location:', this.location.x, this.location.y);
        }
}

const human = new Human();
human.walk(10, 10);
```

This will probably trip you up the first few times you write a class, since we are so used to the object literal syntax. With the introduction of classes, we also get the ability to extend other classes. We can do this with the `extends` keyword. Consider the game we are making, perhaps we want to use the `Human` class we defined earlier as a base for a `Warrior` class.

```
class Warrior extends Human {}
```

This will create a new class base on `Human`. It will have the same `walk` method and properties. However we might want to change the `strength` of the `Warrior`, and to do this we need to adjust the constructor.

super()

If we tried to just call our constructor with some new properties we would see an error.

```
class Warrior extends Human {
        constructor() {
                this.strength = 15;
        }
}

const conan = new Warrior(); //"ReferenceError: this is not defined
```

In order for us to add to our `Warrior` class we need to call `super()` before any definitions with the `this` keyword.

> more explanation for introduction of super()

```
class Warrior extends Human {
        constructor() {
                super();
                this.strength = 15;
        }
}

const conan = new Warrior();
conan.walk(100, 100);
console.log('Strength:', conan.strength);
```

Calling `super()` will call the base class' `constructor()` method and allow us to override properties. This can also be done from other methods in the code.

Static Methods

In order to access methods from a class we have to instantiate it in order to create a new version of it. However we can also create static methods for our classes that allow us to call them without having to instantiate a new object. To do this we can use the `static` keyword in front of the method name.

```
class Human {
        static sayHello() {
                return 'Hi there!';
        }
}

console.log(Human.sayHello());
```

You can mix this `static` method with the `get` syntax that binds a function or property. If you are familiar with this syntax from Objects, it works the same way.

```
class Human {
        static get sayHello() {
                return 'Hi there!';
        }
}

console.log(Human.sayHello);
```

But I read I should never use a class...

There are a lot of options on the web, not sure if you were aware of that! A lot of people state that adding classes to ES6 was a mistake. I wanted to add it to this book and my video series because I wanted to show you things without opinions around them. Now that you have an understanding of how they work, I would like you to form your own opinions about it. It is always best to have your own opinion rather than that of someone you read about.

Map

WE'LL COVER THE FOLLOWING

- Why new data structures
- Map
- Size
- Setting multiple properties
- Map Keys
- Map Methods
  - has()
  - forEach()
  - keys()
  - values()
  - entries()
  - delete()
  - clear()
- WeakMap
  - No enumeration
- Additional Resources

ES6 introduces a few new ways to store data. Up until now we have had Objects and Arrays to store information. In ES6 we now have Map, WeakMap, Set and WeakSet. These Objects provide a new way to store data in a collection of either key value pairs, or collection of values.

Why new data structures

Most languages will have data structures that are purely for data; Ruby has Hashes, Python has Dictionaries. The goal of these structures is to create a

simple data store. In JavaScript we have always used Objects to do this.

Objects can be simple key value pairs, or include more features, such as methods.

However with Objects in JavaScript comes lots of added data that we might not want. When you create a simple object:

```
const person = {};
```

This will create an object that has a prototype chain, it comes with a `__proto__` object that contains inherited methods. We can create Objects that don't have this chain by using the `Object.create()` method to create a simple empty object.

```
const person = Object.create(null);
```

Doing so will create an object with no methods or properties at all. Let's look at how we can use `Map` and `Set` to work with data. This chapter will be about Map, and in the next chapter we will look at Set.

Map

In ES6 a Map is a key value object that we can instantiate to store data. Maps can use any value as a key in the object, and this includes other objects which we will see a little later.

```
const person = new Map();
```

In order to add values to our new Map we can use the `.set()` method.

```
person.set('name','Ryan Christiani');
person.set('age',31);
```

To retrieve values from our Map we use the `.get()` method.

```
console.log(person.get('name')); //Ryan Christiani
```

Size

Something that is pretty nice about Maps is that they allow us to get the size of them by using the `.size` property. This is useful especially if you have ever tried to so something like this in the past on an Object.:

```
const person = {
  name: 'Ryan Christiani',
  age: 31
};

console.log(person.length); //undefined
```

Getting the size of an Object's keys in JavaScript requires us to write some code to count or keep track of the keys. The `.length` property is only for Arrays, but with Maps we can use the `.size` property to get the size back.

```
const person = new Map();
person.set('name', 'Ryan Christiani');
person.set('age', 31);

console.log(person.size); //2
```

Setting multiple properties

If you want to define a Map with multiple properties we need to use a multi dimensional array. The first element in the sub array is the key for your property, the second is the value.

```
const person = new Map([['name','Ryan'],['age',31]]);
console.log(person.get('name')); //Ryan
```

Map Keys

Map Keys

One neat feature of a Map is that you can use just about anything as a key, that means any of the primitive type as well as functions and objects. Meaning you could use something like this as a key.

```
const student = {
  id: '17UsJ290'
};

const person = new Map();
person.set(student,'Ryan Christiani');
console.log(person.get(student)); //Ryan Christiani
```

This can also be done with a function. As a quick example, check out the following.

```
const myFunction = () => {};

const person = new Map();
person.set(myFunction, 'A function used as a key');
console.log(person.get(myFunction)); //A function used as a key
```

I am interested to see how the community ends up using the ability to store values with a function or object as a key; perhaps some work around memoization for functions.

Map Methods

There are quite a few handy methods on Maps that we can use. We will look at `.has()`, `.forEach()`, `.keys()`, `.values()`, `.entries()`, `.delete()`, and `.clear()`.

has()

The `.has` method is used to check for the existence of a value in a key. Using the `person` example from above, let's check to see if it has `age`.

```
console.log(person.has('age')); //true
console.log(person.has('height')); //false
```

forEach()

We can also use the `.forEach()` method to enumerate over the values in our Map.

```
person.forEach((value,key) => {
    console.log(value);
    console.log(key);
});
```

The `.forEach()` method takes a callback that is provided with the `value` and `key` for each element in the Map. It can also accept a second argument that will be used as the `this` context.

keys()

`.keys()` is a method that returns an Iterator Object which allows us to get the keys and use a `for...of` loop to iterate over them.

```
for(let key of person.keys()) {
  console.log(key);
}
```

Alternatively we can store the Iterator Object in a variable and use the `.next()` method to walk through it one at a time.

```
const personKeys = person.keys();
console.log(personKeys.next().value); //name
console.log(personKeys.next().value); //age
console.log(personKeys.next().value); //undefined
```

Once you exhaust the list it will return a value of `undefined`.

values()

Similar to `.keys()`, the `.values()` method will allow us to create a new Iterator Object and either use the `for...of` or `.next()` methods to step through it. However, unlike `.keys()` this will instead return the value stored.

```
for(let value of person.values()) {
  console.log(value);
}
```

And just like `.keys()` we can use the `.next()` method as well.

```
const personValues = person.values();
console.log(personValues.next().value); //Ryan Christiani
console.log(personValues.next().value); //31
console.log(personValues.next().value); //undefined
```

entries()

If you want to have easy access to both the key and value, the `.entries()` method is what you are looking for.

```
for(let property of person.entries()) {
    console.log(property);
}
//["name", "Ryan Christiani"]
//["age", 30]
```

This will return an array where the first element is the key and the second is the value. We can use array destructuring to get the key and value separately if needed!

```
for(let [key,value] of person.entries()) {
  console.log(key);
  console.log(value);
}
```

And just like the last two methods, we can create an Iterator Object that will allow us to step through the properties with the `.next()` method.

```
const personEntries = person.entries();
console.log(personEntries.next().value); //["name", "Ryan Christiani"]
console.log(personEntries.next().value); //["age", 30]
```

delete()

The `.delete()` method takes a key name and will delete this from the Map, and will also return `true` if it is deleted.

```
// Before
console.log(person.get('name'));

person.delete('name'); //true

// After
console.log(person.get('name'));
```

clear()

We can also clear out the entire Map using the `.clear()` method.

```
// Before
console.log(person.get('name'));

person.clear();

// After
console.log(person.get('name'));
```

WeakMap

There is a type of Map that is very similar to what we talked about above, however there is a very key difference. A `WeakMap` allows us to store the same sort of key value pairs as a regular map, but is stores them 'weakly'. This means that when it is not being referenced the garbage collector in the JavaScript engine will clean it up.

If you are not familiar with garbage collection, or GC, it is a process in with the JS engine will attempt to free up memory. When a variable or bit of data is being used and referenced throughout your program the GC can't do anything about it. However when you stop referencing it there is a certain time when it has no purpose, so a garbage collector will come along and clean it up for us. That is an overly simplistic explanation of GC, in the Addition Resources section you can find a link to an MDN article on Memory Management for a more in depth look.

No enumeration

One other thing to look at when it comes to a `WeakMap` is that is can not be enumerated, this means we can not used the `.entries`, `.keys`, or `.value` methods to get a list of information. If this is something you need, just use a Map!

Additional Resources

- http://www.ecma-international.org/ecma-262/6.0/#sec-map-objects
- https://developer.mozilla.org/en/docs/Web/JavaScript/Reference/Global_Objects/Map
- http://www.2ality.com/2015/01/es6-maps-sets.html
- https://developer.mozilla.org/en/docs/Web/JavaScript/Reference/Global_Objects/WeakMap
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_Management

Set

WE'LL COVER THE FOLLOWING

- add()
  - Adding multiple values
- has()
- forEach()
- entries()
- keys()/values()
- delete()
- clear()
- Unique Values only
- WeakSet
- Additional Resources

In the previous chapter we looked at the Map object, and how we can use it to store key/value pairs. In this chapter we will look at Set and how to store unique values.

A Set is created in the same way as Map.

```
const shoppingList = new Set();
```

add()

With a Set we can use the `.add()` method to add elements to the Set. This is where a Map and Set differ: there are no keys in a Set.

```
shoppingList.add('Milk');
shoppingList.add('Cheese');
```

```
shoppingList.add('Cheese');
```

Adding multiple values

When we create a new Set we can pass in an array of values for our Set.

```
const shoppingList = new Set(['Milk','Cheese']);
```

has()

The `.has` method can be used to check if a value exists in the Set.

```
console.log(shoppingList.has('Milk')); //true
```

forEach()

There is no `.get()` method for a Set, so we have to use either a `.forEach()` or an Iterator Object to get access to them. The `.forEach()` behaves as expected: it takes a callback function that is provided three arguments. Oddly, since there are no keys in a Set, the first two are the values from the Set. The last is the the original Set being iterated over.

```
shoppingList.forEach((value) => {
 console.log(value);
});
```

entries()

Much of the Set API matches that of Map. A Set also has an `.entries` method which returns an Iterator Object, and we can use a `for...of` and the `.next()` methods to iterate over it.

```
for(let [value,sameValue] of shoppingList.entries()) {
  console.log(value);
  console.log(sameValue);
}
```

Again, since a Set has no key, the `.entries()` method returns the same values.

```
const shoppingEntries = shoppingList.entries();
console.log(shoppingEntries.next().value); //["Milk","Milk"]
console.log(shoppingEntries.next().value); //["Cheese","Cheese"]
```

keys()/values()

Similar to Maps, we can use the `.keys()` and `.values()` methods to access the elements in our Set. However since there are no keys in a Set both of these methods will return an Iterator Object that will provide the values in the Set.

delete()

Again, much like a Map, we can delete a property with the `.delete()` method, in this case we can just pass the value we want to remove from our Set.

```
shoppingList.delete('Milk');
console.log(shoppingList.has('Milk')); //false
```

clear()

If we need to completely wipe our Set, we can use the `.clear()` method.

Unique Values only

A Set can only contain unique values, so if you try to add a duplicate it will do nothing.

```
const shoppingItems = ['Milk','Bread','Cheese','Chips','Milk','Peaches'];
const uniqueShoppingItems = new Set(shoppingItems);
```

```
for(let item of uniqueShoppingItems.values()) {
  console.log(item);
}

//Milk
//Bread
//Cheese
//Chips
//Peaches
```

This is a great little trick for if you have an array of duplicate vales, and you want to get the unique values out of it.

WeakSet

Just like a `Map`, the `Set` has a weak alternative. However there is a key difference here. For a `WeakSet`, you are only allowed to store objects, not any other value.

```
const weak = new WeakSet(['test']); //TypeError: Invalid value used in weak set
```

```
const weakObject = {};
const weakObjCollection = new WeakSet([weakObject]); //Will throw no error
console.log(weakObjCollection);
```

Just like a `WeakMap`, it will hold on to these values weakly, meaning when they are not referenced anymore, they will be garbage collected.

Additional Resources

- https://developer.mozilla.org/en/docs/Web/JavaScript/Reference/Global_Objects/Set
- https://developer.mozilla.org/en/docs/Web/JavaScript/Reference/Global_Objects/WeakSet

Promises

WE'LL COVER THE FOLLOWING

- all()
- race()
- fetch()

Promises have been around for a while in JavaScript, but we have never had them in the language itself. In the past we would have had to use a library like `q` or `bluebird`. These libraries follow something called Promises/A+ , it is a standard that outlines how a Promise should behave, and it is the spec that ES6 Promises also implement.

A Promise represents the eventual result of some sort of asynchronous operation. Commonly these are used in conjunction with an HTTP request. However, it could actually just be any sort of event that will take some time to return. Promises can be a few different states. When a promise is created it has a 'pending' state. From there the Promise can have 2 other states, it can be fulfilled(commonly you will hear the word resolved used here), or rejected.

In ES6 the Promise is a constructor that when instantiated will return a new object. This constructor takes what is called an executor, which is a callback function that has two parameters.

```
const myPromise = new Promise((resolve,reject) => {

});
```

In our executor we are provided with a `resolve` and a `reject` function that allow us to control the promise. Let's go ahead and make our first Promise and then resolve it.

```
const myPromise = new Promise((resolve, reject) => {
    resolve('Good to go');
});
```

Here we are resolving immediately. We store the Promise on the `myPromise` variable which has an object that has a few methods on it, one of the most important being `.then`. The `.then` method is how we can see what has happened with our Promise, and it takes two callback functions itself. The first is what happens when a promise resolves successfully.

```
const myPromise = new Promise((resolve, reject) => {
    resolve('Good to go');
});

myPromise.then((res) => {
    console.log(res); //Good to go
});
```

The second would be if our Promise was rejected. Let's change our code to see this in action.

```
const myPromise = new Promise((resolve, reject) => {
    reject('Not good to go');
});

myPromise
  .then((res) => {
  //will not be called
  console.log(res);
}, (err) => {
  //will be called
  console.log(err); //Not good to go
});
```

Here we reject the Promise, and in this case the second callback from the `.then` method will be called.

There is an easier way to catch a rejected Promise, and that is with the `.catch` method. We could change the above code to look like this.

```
const myPromise = new Promise((resolve, reject) => {
    reject('Not good to go');

});

myPromise
  .then((res) => {
  //Only if the promise resolved
  console.log(res);
})
  .catch((err) => {
  console.log(err); //Not good to go
});
```

Using the `.catch` method we can handle the rejected promise with ease. Since a promise is all about the eventual effects of some action, lets add a `setTimeout` to simulate that eventuality of an asynchronous event.

```
const myPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve('Good to go')
  }, 1000)

  setTimeout(() => {
    reject('Uh ohh');
  },1500)
});

myPromise
  .then((res) => {
  console.log(res);
})
  .catch((err) => {
  console.log(err);
});
```

Looking at the above, we have two `setTimeout` functions that will run the `resolve` and `reject` functions at different times. From the code example above the `.then` success will be called since it resolves first. I should point out that once a Promise is either fulfilled or rejected its state can no longer change, so in this case the `Uh ohh` will never be seen.

all()

What about dealing with multiple Promises? Promises in ES6 has a static method called `.all` on it, meaning that we do not have to instantiate a new

Promise object, but rather we can just use the method straight up. First let's create another promise.

```
const myPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve('Good to go')
  }, 1000)

  setTimeout(() => {
    reject('Uh ohh');
  },1500)
});

const promiseSequel = new Promise((resolve,reject) => {
  setTimeout(() => {
    resolve('Promise 2, the promising');
  },500);

  setTimeout(() => {
    reject('Box office bomb');
  });
});

Promise.all([myPromise,promiseSequel])
  .then((...resolvedData) => {
  console.log(resolvedData);
})
  .catch((...errors) => {
  console.log(errors);
});
```

The `.all` method will take an iterable object (in our case an array) and wait until all of the Promises resolve. If even ONE of them rejects the `.catch` or second callback from `.then` will be called.

race()

The Promise object also has a method called `.race`, this is similar to `.all` however it will only be called once and this is when any of the provided promises either resolves or rejects. So if you just need any data as fast as possible, the `.race` method is for you.

fetch()

Something we should look at is `fetch` so that we can see this in a real world

Something we should look at is `fetch` so that we can see this in a real world situation. Fetch is a new API in the browser that allows us to make HTTP

requests, and it uses ES6 Promises under the hood.

Fetch takes a URL as the first argument and an object as the second that allows us to configure the request. For this example we can just pass the URL.

The fetch function returns a Promise so we can use the `.then` method on it. The value provided to

```
import "isomorphic-fetch"

fetch('http://pokeapi.co/api/v2/pokemon/1/')
  .then((res) => {
  return res.json()
})
  .then((data) => {
  console.log(data)
})
.catch((err) => {
  console.log(err);
});
```

Generators

WE'LL COVER THE FOLLOWING

- Our First Generator
- yield*
- Passing a value to .next()
- Synchronous asynchronous calls
  - coro step by step
  - Additional Resources

ES6 brings us a new type of function, the generator function. Generators are a function that allows us to pause and resume execution. Generators exist in many other languages and are used mostly for control flow.

The syntax to create a generator is the same as a regular function, however we use the `*` at the end of the `function` keyword to indicate we are creating a generator.

```
function* functionName() {

}
```

Generators return an Iterator Object, similar to Map, Set, and Array. An Iterator Object is an object that implements a `.next()` method. This means that our generator function can provide multiple results. There is also a new keyword introduced called `yield`, the `yield` keyword is used to actually pause the execution of the function and return the value provided.

Our First Generator

Let's look at a simple example of a generator.

```
function* myGenerator() {
    yield "Isn't";
    yield "this";
    yield "cool!";
}
```

Inside of the generator we are able to `yield` as many values as you want. Here we are yielding three times, in order to get at those values we need to call our generator.

```
const generatorValues = myGenerator();
console.log(generatorValues.next());
/*
{
    value: "Isn't", done: false
}
*/
```

The next method returns an object that has the two keys on it, `value` and `done`. Until our generator has finished, the `done` value will always be false, and the `value` will be what we yield. Calling `.next()` three more times will look something like this.

```
console.log(generatorValues.next());
//{ value: 'this', done: false }

console.log(generatorValues.next());
//{ value: 'cool!', done: false }

console.log(generatorValues.next());
//{ value: undefined, done: true }
```

After we have exhausted our generator we can call it all we want, but it will only return the last object with `done` set to `true`. With this information in mind, let's look at a more complex example.

```
function* countUp(iterations) {
```

```
    let counter = 0;
    while(counter < iterations) {
        yield counter++;

    }
}
```

The `countUp` generator function here takes an argument that will be used as the number of iterations we want to go for. Using a `while` loop we can `yield` until we exhaust our iterations.

Remember that calling our generator function returns an Iterator Object. This means we can use the `for...of` loop to iterate over it.

```
for(let iteration of countUp(5)) {
  console.log(iteration);
}
// 0
// 1
// 2
// 3
// 4
```

Here instead of it providing the object with the `done` and `value` keys, it simply provides the value for us in the `iteration` variable.

yield*

Along with the regular `yield` keyword we can use the `*` as well to accept a value that it's self is an Iterator Object.

```
const locations = new Set(["Toronto","Montreal","New York"]);

function* locationGen() {
  yield* locations;
}

const myLocations = locationGen();

console.log(myLocations.next());
//{ value: 'Toronto', done: false }

console.log(myLocations.next());
//{ value: 'Montreal', done: false }
```

The `yield*` syntax will iterate over the object provided, in this case we are using a `Set` as the Iterable Object.

Passing a value to .next()

When working with the `yield` keyword we can also pass a value to the generator through the `.next()` method. Consider the `add()` function below.

```
function* add() {
  let total = 0;
  while(true) {
    let value = yield;
    total += value;
    console.log(total)
  }
}
```

Here we have a generator function called `add` that will have a running total. There is a while loop that will run forever if left to do so, however since this is a generator it will pause at the `yield` keyword.

We assign `yield` to a new `value` variable and add that to the total. Below you can see how we might call this.

```
const addFn = add();
console.log(addFn.next()); //Used to call the generator up to the yield
console.log(addFn.next(2)); //2 will be passed to yield and added to total
console.log(addFn.next(2)); //2 will be added to 2
console.log(addFn.next(2)); //2 will be added to 4
```

Now that we have seen how to pass a value back to a generator, let's see how we can use generators to create synchronous asynchronous calls.

Synchronous asynchronous calls

Working with asynchronous JavaScript can be complex, Promises have made this easier, and in the future `async/await` will help a lot. But there is a neat pattern that we can use now. This is a pattern I have seen on Axel Rauschmayer site, as well as talked about by Kyle Simpson. The code we will

be working with is similar that talked about by Kyle. This pattern is also implemented in a library by TJ Holowaychuk, https://github.com/tj/co.

We will be implementing a coroutine function that will allow us to call multiple async calls and get them back in the order we called them. Take a look at the code below.

```
const pokemon = coro(function* () {
    let pokemon1 = yield getPokemon(1,pokemon);
    let pokemon2 = yield getPokemon(242,pokemon);
    console.log(pokemon1,pokemon2);
});
pokemon();
```

Here we use a function called `coro` that takes a generator callback function. This function can yield as many times as we want. We yield a function called `getPokemon` that takes a pokemon id and takes the `pokemon` function so that we can pass the data back.

The `getPokemon` function looks something like this, we are using `fetch` to call the pokeapi and return the json version of the response. We run the callback and pass the response back.

```
function getPokemon(id,cb) {
    fetch(`http://pokeapi.co/api/v2/pokemon/${id}`)
        .then(res=>res.json())
        .then(res=>cb(res));
}
```

The `coro` function looks like this. We take in a generator function and call it. We return a function that itself returns the `.next()` method and apply the original callback function and the arguments we pass to it.

```
function coro(g) {
    let it = g();
    return function() {
        return it.next.apply(it,arguments);
    }
}
```

coro step by step

Let's walk through the `coro` function and how it works, since it is a little complex. We create a new function called `pokemon`, using the `coro` function we pass in a generator to use as our control flow. Inside of the `coro` function it takes the generator and returns a function that returns a call to the `next()` method and it passes in the generator again, as well as any arguments provided. In the first case it will pass along no data as no arguments have been passed to the function.

When it gets to this line:

```
let pokemon1 = yield getPokemon(1,pokemon);
```

We are making a called to the `getPokemon` function to make our ajax call. We pass `pokemon` back to it so that when the `getPokemon` call resolves it will pass that data to `yield` ultimately storing it in the `pokemon1` variable. We could do this over and over, each time the when we get to a new line, we wait for the data to come back before processing on.

I would suggest trying this code out in your browser, the whole thing looks like this:

```
import "isomorphic-fetch"

function getPokemon(id,cb) {
    fetch(`http://pokeapi.co/api/v2/pokemon/${id}`)
        .then(res=>res.json())
        .then(res=>cb(res));
}

function coro(g) {
    let it = g();
    return function() {
        return it.next.apply(it,arguments);
    }
}

const pokemon = coro(function* () {
    let pokemon1 = yield getPokemon(1,pokemon);
    let pokemon2 = yield getPokemon(242,pokemon);
    console.log(pokemon1,pokemon2);
});

pokemon();
```

Try walking through your code with a `debugger` or simply some well placed `console.log` calls to watch how the data flows.

Additional Resources

- https://tc39.github.io/ecmascript-asyncawait/

Modules

WE'LL COVER THE FOLLOWING

- Export
  - Named exports

Before we get started looking at Modules, we need to change the Gulp and Babel files that we set up earlier to allow us to work with modules. We need to add four new modules: `browserify`, `babelify`, `vinyl-source-stream`, and `vinyl-buffer`.

Support for Modules in browsers does not quite exist in the browser. Even though ES6 introduces modules in the language so we need to use something to transpile and bundle them. There are many popular module bundler's out there, in this book we will look at Browserify.

Browserify is a popular module loader. It allows you to take npm modules, or our own modules, and bundle them up into a single file! It takes the `require` functions from node and, as they say "recursively analyze all our dependences" in order and serve up a single file.

However if we are going to use the ES6 `import` and `export` keywords we need to first transform our code. Enter Babelify, this is a transformer for Browserify that will take our ES6 and make it work well together.

The `vinyl-source-stream` and `vinyl-buffer` modules are used to take the output from Browserify and gather it all up so that we can put it all in one file. The output is a stream, and we can used these to property pull everything together.

To get started make sure you have read the Using ES6 now chapter, as we will have a similar set up. You can also find the files for this and the previous

example on GitHub at https://github.com/lets-learn.

Make sure you navigate towards a folder you would like work in, to get started let's install our npm modules.

```
npm install --save-dev browserify babelify vinyl-buffer vinyl-source-stream
```

With these installed let's start changing the gulp file from the previous chapter.

```
const gulp = require('gulp');
const babelify = require('babelify');
const browserify = require('browserify')
const source = require('vinyl-source-stream');
const buffer = require('vinyl-buffer');
```

In our ES6 task we need to remove the current code and change it to look like this.

```
gulp.task('es6', () => {
  browserify('src/app.js')
    .transform('babelify', {
    presets: ['es2015']
  })
    .bundle()
    .pipe(source('app.js'))
    .pipe(buffer())
    .pipe(gulp.dest('build/'));
});
```

We now use `browserify` to take our source file, look for an modules in there, and get them! We transform it with `babelify`. This step is very similar to our original gulp file, where we set the preset to be `es2015`. Remember that Babel will not transpile our code from ES6 by default, so we need to make sure we include the `babel-preset-es2015`.

Next we need to bundle up our files. Here we call the `.bundle` method and pass that into our `vinyl-source-stream`. The `.bundle` method returns a stream so we use `vinyl-source-stream` to handle that stream, and `vinyl-buffer` takes that stream and creates an output file for us.

Export

Let's create our first module, there are a couple ways to create modules. Modules can have a single `default` export, or a module can have multiple named exports.

Start by creating a file called `myFirstModule.js`.

```
export default function(text) {
  console.log(`My first module is ${text}`);
};
```

We have two keywords here, `export` and `default`. `export` is needed to define that we are going to be exporting something, here we are setting up the `default` exports. A file can have multiple exports, but we can also specify a `default` to act as the fall-back.

Now that we have a simple default export set up, let's look at how we can import it. Create a new file called `app.js` and also create an `index.html` file to load our script. In the `app.js` file we can import our new module like this.

```
import React from 'react';
import first from './myFirstModule';

console.log(first('Neat!'));
```

In the `app.js` file we use the `import` keyword to declare our import, we then give it a name, in this case we are calling it `first`. We could name this anything we like. The `from` keyword is used to set the modules source location, in our case we are using a file in our directory.

With Gulp running, open that file in the browser, and you should see your text in the console.

Notice how we can leave the `.js` off, it is assumed that it will be a `.js` file, so we can exclude the extension.

We can also import npm modules, say we installed `react` via npm. We could import it like such.

```
import React from 'react';
```

Named exports

A file can only have one default export, but it can have many named exports. To create a named export we leave the `default` keyword off and supply a function or variable with a name. Create a new file called `myMathModule.js`;

```
export function add(a,b) {
  return a + b;
};
```

One way to import this named export is to use destructoring to get it. When we have named exports our modules now exports an object. Using destructoring we can pick the properties we want. If you need a refreshing on destructoring check out the destructoring chapter.

```
import { add } from './myMathModule';

const total = add(4,5);

console.log(total);
```

Let's add another export to see how this works.

```
export function add(a,b) {
  return a + b;
};

export function subtract(a,b) {
  return a - b;
};
```

Above we have added a `subtract` function, now if we wanted to import this in another file we can do this.

```
import { add, subtract } from './myMathModule';
```

This is great because we don't have to include all the modules if we don't need to use them. And tools like Rollup, which performs a method called tree shaking, will see this on only bundle the code used, not the entire module.

We can also pull in all of our named exports using the `*` character and the `as` keyword.

```
import * as myMath from './myMathModule';

myMath.add(2,3); //5
myMath.subtract(10,7); //3
```

One other thing to note about a module is that it can have named exports along with a default export. Let's change our `myMathModule` to include a default as well.

```
export function add(a,b) {
  return a + b;
};

export function subtract(a,b) {
  return a - b;
};

export default function (a,b) {
  return a / b;
};
```

Here we added a default export that acts as our divide function, now we can import this and our `add` by defining the default first, followed by a `,` and then the destructored `add`.

```
import divide, { add } from './myMathModule';
```

Modules are a great way for developers to break up their code into more manageable and organized pieces. They allow us to create maintainable code that is less daunting to get work with.

```
export function add(a,b) {
  return a + b;
};

export function subtract(a,b) {
  return a - b;
};

export default function (a,b) {
  return a / b;
};
```

Proxies & Reflect

WE'LL COVER THE FOLLOWING

- One step further with proxies
- Reflect
  - Checking for a key
  - Additional Resources

A Proxy in ES6 is a special type of object that allows us to intercept operations preformed on an Object. Operations like property lookups, assignments and more. Proxies all us to set up a `target` object with a `handler`. The `handler` object is used to override the operations on our target object. Let's first look at the syntax for our Proxy.

```
new Proxy(target,handler);
```

With this in mind, let's create a `person` object with two simple properties on it.

```
const person = new Proxy({
    name: 'Ryan',
    age: 31
},{
    //Handler goes here.
})
```

On this `person` object, we have only two properties `name` and `age`. The `handler` object is used to list 'traps', these trap methods include `get()`, `set()`, and much more. As an example, lets set up a Proxy that has a `get` trap.

```
const person = new Proxy({
  name: 'Ryan',
  age: 31
}, {
```

```
  get(target, prop) {
    return target[prop]


  }
});
```

Here we implement a `get` trap on our handler, notice that the method gets passed two arguments, `target`, and `prop`. `target` is the object we are trying to access the property from. And `prop` is what `key` we want to access. Calling `person.name` will trigger the `get` trap, as we are trying to access the `name` property. In our trap here we simply return the value, but image that we could alter the values.

Let's change this to return all uppercase strings from our object.

```
const person = new Proxy({
  name: 'Ryan',
  age: 31
}, {
  get(target, prop) {
    const val = target[prop];
    return typeof val === 'string' ? val.toUpperCase() : val;
  }
});

console.log(person.name); //RYAN
```

This opens up a lot of possibilities, because now we have the ability to intercept operations on our objects and validate or alter them. We have had the ability to do this in the past with the `get` and `set` keywords, remember we looked at `get` a little bit back in the chapter on classes. However with those we had to create a `get` and a `set` for each property, here we are able to do that with more programmatically as we are passed all the info we need.

We are just scratching the surface of Proxies, I will list of bunch of great resources at the end of the chapter so a more indepth look into them. But to finish, let's implement a little `Component` function that we can pass some data, and validate the data when we want to change it. Also using the `set` trap we will have it re-render our `Component` when data is `set`.

To start, we will use the `templater` function from chapter 7 to create our HTML

HTML.

```
//From chapter 7
function templater(strings, ...keys) {
    return function(data) {
        let temp = strings.slice();
        keys.forEach((key, i) => {
            temp[i] = temp[i] + data[key];
        });
        return temp.join('');
    }
}
```

Next we need to create our `Component` function, it will take an `options` object and return a slightly altered version of it. Ideally this is how we would like to call a new `Component`.

```
const app = Component({
  model:{
    num:'0'
  },
  template: templater`<h1>Clicked: ${'num'}</h1>`
});
```

The important bit here is the `model` property, this we will convert into a Proxy so we can intercept the `set` operation. Now we will look at the `Component` and talk about what is going on here.

```
function Component(options) {
  let defaults = {
    model: new Proxy(options.model,{
      set(target,prop,value) {
        target[prop] = value;
        defaults.render(target);
      }
    }),
    template: options.template,
    el: options.el || document.body,
    render(data) {
      this.el.innerHTML = this.template(data)
    }
  }
  defaults.render(options.model);
  return defaults;
}
```

First thing you notice is we create a `defaults` object that is used to set us up. We have a `model` key that we pass a `new Proxy` to, that takes the `model` passed from `options` and sets the `set` trap. This trap is provided three arguments.

the `target`, the `prop` and what `value` we want. In the trap we set the value, but also we call the `render` method of the `defaults` object.

If we look at `render` we can see it takes some data, our object, and used the `el` property(which is `body` by default) and renders our template on it. The really neat thing here is that if we create a new `Component` and then have an event listener that every time the `body` is clicked we update the value on `num`, it will re render our view!

```
const app = Component({
  model:{
    num:'0'
  },
  template: templater`<h1>Clicked: ${'num'}</h1>`
});

let counter = 0;

document.addEventListener('click',() => {
    counter++;
    app.model.num = counter;
});
```

Kind of fun right? I mean this is no React, but it gets you thinking about how intercepting object operations can help us create intuitive API's. This concept is called meta programming, and in the *Additional Resources* section I have added a link from Axel Rauschmayer on Reflection and Meta Programming for a further read. There are also many more traps we can add, to see the whole list, check out the MDN article listed below.

One step further with proxies

Let's take this idea one step further and implement type validation to our `model`. Suppose we want to make sure the user is always setting a certain type for data model, like a number for example.

```
const app = Component({
  model:{
    num: '`number` 0'
  },
  template: templater`<h1>Clicked: ${'num'}</h1>`
});
```

We can use the initial setup of our `Component` to create an object that can be used for validation.

```
const props = Object.assign({},options.model);
const model = {};
for(let key in props) {
 let type = /`(.*)`/gi.exec(props[key])[1];
 let value = /^.*`\s(.*)/gi.exec(props[key])[1];
 if(type === 'number') value = parseFloat(value)
 props[key] = {
   type,
   value
 }
 options.model[key] = value
}
```

There is some regExp here that looks for anything in the `` and will use that as the type, and the it will set the value back on the options `model`. In our `set` trap we can add a condition that checks the `value` that comes in, if it is not right it will throw an error.

```
set(target,prop,value) {
  if(typeof value !== props[prop].type) {
    throw new Error(`Invalid type set on property ${prop}`);
  }
  target[prop] = value;
  defaults.render(target);
}
```

Below you can find the whole code.

```
function templater(strings, ...keys) {
    return function(data) {
        let temp = strings.slice();
        keys.forEach((key, i) => {
            temp[i] = temp[i] + data[key];
        });
        return temp.join('');
    }
}

function Component(options) {
  const props = Object.assign({},options.model);
  const model = {};
  for(let key in props) {
    let type = /`(.*)`/gi.exec(props[key])[1];
    let value = /^.*`\s(.*)/gi.exec(props[key])[1];
    if(type === 'number') value = parseFloat(value)
    props[key] = {
      type,
      value
    }
    options.model[key] = value
```

```
  }
  let defaults = {
    model: new Proxy(options.model,{

      set(target,prop,value) {
        if(typeof value !== props[prop].type) {
          throw new Error("Not the right type")
        }
        target[prop] = value;
        defaults.render(target);
      }
    }),
    template: options.template,
    el: options.el || document.body,
    render(data) {
      this.el.innerHTML = this.template(data)
    }
  }
  defaults.render(options.model);
  return defaults;
}


const app = Component({
  model:{
    num: '`number` 0'
  },
  template: templater`<h1>Clicked: ${'num'}</h1>`
});

let counter = 0;

document.addEventListener('click',() => {
    counter++;
    app.model.num = counter;
});
```

Reflect

Reflect is a new object in JavaScript that contains most of the methods that the Object has on it, however it allows us to use these methods with more reliability. When looking to find good examples of Reflect, I stumbled across a Stack Overflow that was great, you can find it in the list of additional resources below.

The goal of `Reflect` is to provide static methods that match with the proxy traps we just went over. The idea is that we can use the `Reflect.apply` method with out having to worry if our object that we want to use it on has implemented its own `.apply` method. It also allows us to work with our objects with more ease.

Checking for a key

In JavaScript if we want to check for a key on an Object we typically do something like this `key in myObject`, now we can is the `Reflect.has()` method that will return `true` or `false`.

```
const person = {
  name: "Ryan",
  age: 31
};

if('age' in person) {
  console.log('Age!')
}

if(Reflect.has(person, 'age')) {
  console.log('Relect age!')
}
```

There are a lot of other use cases for the `Reflect` object, however I do not want to rehash some of the great information out there on the Internet, so listed in the additional resources, I have added a bunch of great links.

Additional Resources

- https://developer.mozilla.org/en/docs/Web/JavaScript/Reference/Global_Objects/Proxy
- http://www.2ality.com/2011/01/reflection-and-meta-programming-in.html
- https://ponyfoo.com/articles/es6-reflection-in-depth
- http://stackoverflow.com/questions/25421903/what-does-the-reflect-object-do-in-javascript
- https://github.com/tvcutsem/harmony-reflect/wiki

Symbols

WE'LL COVER THE FOLLOWING

- Symbols for private keys
- Well known symbols
  - Additional Resources

Symbols are a new primitive data type in JavaScript, they can to create anonymous keys for object properties, among other things. One key feature of these properties is that they do not show up in a `for..in` loop. Symbols are created using the `Symbol` function and passed a description.

```
const mySym = Symbol('name');
```

No two symbols are the same, if we created another `Symbol('name')` it would be completely different.

Symbols for private keys

Because of a Symbols ability to be unique and excluded from enumeration, it is common to use them to create private properties. Because JavaScript has no `private` keyword this is a great use case for the `Symbol`.

Let's assume we made a `Superhero` class in a module, we can use a `Symbol` to create a `_secretIdenity` property to be used in the class.

```
//Super hero modules
const _secretIdenity = Symbol('_secert');

exports default class Superhero {
    constructor(identity) {
        this[_secretIdenity] = identity;
        this.power = 100;
    }
```

```
    }
    getIdentity() {
      return this[_secretIdenity];

    }
}
```

In our main app we import this class and create a new `Superhero`.

```
//Main application
//import Superhero from 'superhero.js';

const wonderWoman = new Superhero("Diana Prince");
console.log(wonderWoman.getIdentity());
```

We are not able to access the property like we normally would, however using our `getIdentity` method we can get access to the property! If we run this new object through a `for...in` loop you will notice we don't see the Symbol property at all!

```
for(let key in wonderWoman){
  console.log(key);
}
//power
```

This is not perfect however, because we can us the `Object.getOwnPropertySymbols()` method to get a list of the symbols.

```
const theSymbol = Object.getOwnPropertySymbols(wonderWoman);
console.log(wonderWoman[theSymbol[0]]); //Diana Prince
```

But is starts to get us there.

Well known symbols

Symbols can also be used to change how we an iterate through an array. Take this `iterable` array for example, we can use the `Symbol.iterator` property of

this `cities` array for example, we can use the `Symbol.iterator` property of

our array to create a new Iterator Object, remember we talked about Iterators in the Generators Chapter.

```
const cities = ['Toronto', 'Montreal', 'Boston', 'San Fransisco'];
let cityIterator = cities[Symbol.iterator]();

console.log(cityIterator.next());
/*
    { value: "Toronto", done: false }
*/
console.log(cityIterator.next());
/*
    { value: "Montreal", done: false }
*/
console.log(cityIterator.next());
/*
    { value: "Boston", done: false }
*/
console.log(cityIterator.next());
/*
    { value: "San Fransisco", done: false }
*/
console.log(cityIterator.next());
/*
    { value: undefined, done: true }
*/
```

This now gives us a new way to take an existing array and iterate through it. Because of this, we can use this well known Symbol to create an iterator object out of a HTMLCollection.

```
let listItems = document.querySelectorAll('li');
let listIterator = listItems[Symbol.iterator]();

listIterator.next();
```

I am very interested to see how these will be used going forward, for more information about Symbols, please check out the additional resources below.

Additional Resources

- https://developer.mozilla.org/en-US/docs/Glossary/Symbol
- https://developer.mozilla.org/en/docs/Web/JavaScript/Reference/Global_Objects/Symbol

ljects/Number

ES7(ES2016) & Beyond

WE'LL COVER THE FOLLOWING

- Exponents
- Array.includes

There was a lot of things added in ES6, luckily ES7 or ES2016 has one two features to talk about! That's right TWO! Much easier to digest and get caught up on. These features are the Exponents operator and the Array `.includes` method.

Exponents

In JavaScript expressions are evaluated based on BEDMAS, brackets, exponents, division, multiplications, additions, and subtractions. However we have never been able to express an exponent elegantly. There is a method on `Math` that let's us express a base to some exponent.

For example, 2 to the power of 8.

```
console.log(Math.pow(2,8)); //256
```

In ES7 we now have the `**` operator that allows us express this more simply.

```
console.log(2 ** 8); //256
```

That is all you need to know about the exponent operator!

Array.includes

If you want to see if an array has a specific element in it, you have had multiple ways of doing this. You can use `.indexOf`, as we saw in Chapter 9 we can also use `.find`. The `.indexOf` method returns the index of the element if it exists or `-1` if it does not, and `.find` will actually find and return the element. However maybe we just want a simple `true` or `false` to see if something exists or not. Enter the `.includes` method!

```
const cities = ["New York","Toronto","Miami","Montreal"];
console.log(cities.includes("Paris")); //false
console.log(cities.includes("Toronto")); //true
```

The method is that straightforward, given an array you can check to see if an element exists in it or not. There is an optional second argument the method can take, and that is the index of the array that you would like to start from. If no second argument is provided it will default to `0`. Let's add some more cites to our array and see how this works.

```
const cities = ["New York","Toronto","Miami","Montreal","Vancouver","Berlin","Tokyo"];
console.log(cities.includes("Toronto",3)); //false
console.log(cities.includes("Berlin",3)); //true
```

The `fromIndex` argument tells `.includes` to start only from index 3, meaning it starts looking at `"Montreal"`. This will be beneficial if you only want to search the last half of your array for some element.